2026-10-31 16:12:51 You

Practical-1 (Uber)
Problem Statement: Predict the price of the Uber ride from a given pickup point to the agreed drop-off location. Perform following tasks:
1. Pre-process the dataset.
2. Identify outliers.
3. Check the correlation.
4. Implement linear regression and random forest regression models.
5. Evaluate the models and compare their respective scores like R2, RMSE, etc.
Steps
1.Importing Libraries
2.Data Loading and Pre-processing
3.Outlier Detection
4.Correlation Analysis
5.Model Implementation (Linear Regression & Random Forest)
6.Model Evaluation and Comparison
Step 1 : Importing Libraries
Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from math import radians, cos, sin, asin, sqrt
Step 2 : Data Loading & Preprocessing
Load the dataset
df = pd.read_csv("uber.csv") # change to your local path if needed
print("Initial Data Shape:", df.shape)
print(df.head())
Drop rows with missing values
df.dropna(inplace=True)
print("After dropping missing values:", df.shape)
Rename columns for easier reference

```
df.rename(columns={'pickup_datetime': 'pickup_datetime'},
inplace=True)
# Convert pickup_datetime to datetime object
df['pickup_datetime'] = pd.to_datetime(df['pickup_datetime'],
errors='coerce')
# Extract useful datetime features
df['hour'] = df['pickup_datetime'].dt.hour
df['day'] = df['pickup_datetime'].dt.day
df['month'] = df['pickup_datetime'].dt.month
df['year'] = df['pickup_datetime'].dt.year
df['day_of_week'] = df['pickup_datetime'].dt.dayofweek
# Drop datetime column (not needed as a direct feature)
df.drop(['pickup_datetime', 'key'], axis=1, inplace=True,
errors='ignore')
print("\nColumns after feature extraction:\n", df.columns)
Initial Data Shape: (200000, 9)
 Unnamed: 0 key fare_amount \
0 24238194 2015-05-07 19:52:06.0000003 7.5
1 27835199 2009-07-17 20:04:56.0000002 7.7
2 44984355 2009-08-24 21:45:00.00000061 12.9
3 25894730 2009-06-26 08:22:21.0000001 5.3
4 17610152 2014-08-28 17:47:00.000000188 16.0
 pickup_datetime pickup_longitude pickup_latitude \
0 2015-05-07 19:52:06 UTC -73.999817 40.738354
1 2009-07-17 20:04:56 UTC -73.994355 40.728225
2 2009-08-24 21:45:00 UTC -74.005043 40.740770
3 2009-06-26 08:22:21 UTC -73.976124 40.790844
4 2014-08-28 17:47:00 UTC -73.925023 40.744085
 dropoff_longitude dropoff_latitude passenger_count
0 -73.999512 40.723217 1
1 -73.994710 40.750325 1
2 -73.962565 40.772647 1
3 -73.965316 40.803349 3
4 -73.973082 40.761247 5
After dropping missing values: (199999, 9)
Columns after feature extraction:
 Index(['Unnamed: 0', 'fare_amount', 'pickup_longitude',
'pickup_latitude',
 'dropoff_longitude', 'dropoff_latitude', 'passenger_count',
'hour',
 'day', 'month', 'year', 'day_of_week'],
```

```
 dtype='object')
Step 3 : outliers Detection & Removal
# Remove entries with unrealistic fares
df = df[(df['fare_amount'] > 0) & (df['fare_amount'] < 100)]
# Remove unrealistic latitude and longitude values
df = df[(df['pickup_latitude'] <= 90) & (df['pickup_latitude'] >= -
90)]
df = df[(df['dropoff_latitude'] <= 90) & (df['dropoff_latitude'] >= -
90)]
df = df[(df['pickup_longitude'] <= 180) & (df['pickup_longitude'] >= -
180)]
df = df[(df['dropoff_longitude'] <= 180) & (df['dropoff_longitude'] >=
-180)]
print("Data shape after removing outliers:", df.shape)
Data shape after removing outliers: (199875, 12)
Step 4 : Feature Engineering - Distance Calculation
# Define Haversine function to calculate distance between pickup
and
drop-off
def haversine(lat1, lon1, lat2, lon2):
 # convert decimal degrees to radians
 lat1, lon1, lat2, lon2 = map(radians, [lat1, lon1, lat2, lon2])
 # haversine formula
 dlon = lon2 - lon1
 dlat = lat2 - lat1
 a = sin(dlat/2)**2 + cos(lat1) * cos(lat2) * sin(dlon/2)**2
 c = 2 * asin(sqrt(a))
 km = 6371 * c
 return km
# Apply the Haversine formula
df['distance_km'] = df.apply(lambda x: haversine(x['pickup_latitude'],
x['pickup_longitude'], x['dropoff_latitude'], x['dropoff_longitude']),
axis=1)
# Remove zero-distance trips
df = df[df['distance_km'] > 0]
Step 5 : Correlation Analysis
correlation = df.corr(numeric_only=True)
plt.figure(figsize=(10, 6))
plt.imshow(correlation, cmap='coolwarm', aspect='auto')
plt.colorbar(label='Correlation')
plt.xticks(range(len(correlation.columns)), correlation.columns,
```

```
rotation=45, ha='right')
plt.yticks(range(len(correlation.index)), correlation.index)
plt.title("Feature Correlation Heatmap")
plt.tight_layout()
plt.show()
Step 6 : Model Training
# Define features and target
X = df[['distance_km', 'hour', 'day', 'month', 'year', 'day_of_week']]
y = df['fare_amount']
# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
# ---------- Linear Regression ----------
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
y_pred_lr = lr_model.predict(X_test)
# ---------- Random Forest Regression ----------
rf_model = RandomForestRegressor(n_estimators=100,
random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
Step 7 : Model Evaluation
def evaluate_model(y_true, y_pred, model_name):
 r2 = r2_score(y_true, y_pred)
 rmse = np.sqrt(mean_squared_error(y_true, y_pred))
 mae = mean_absolute_error(y_true, y_pred)
 print(f"\nModel: {model_name}")
 print(f"R² Score: {r2:.4f}")
 print(f"RMSE: {rmse:.4f}")
 print(f"MAE: {mae:.4f}")
 return r2, rmse, mae
# Evaluate both models
lr_scores = evaluate_model(y_test, y_pred_lr, "Linear Regression")
rf_scores = evaluate_model(y_test, y_pred_rf, "Random Forest
Regressor")
Model: Linear Regression
R² Score: 0.0172
RMSE: 9.4318
MAE: 5.9225
Model: Random Forest Regressor
R² Score: 0.8190
```

RMSE: 4.0477
MAE: 2.1524
Step 8 : Comparison

```
results = pd.DataFrame({
 'Model': ['Linear Regression', 'Random Forest Regressor'],
 'R2': [lr_scores[0], rf_scores[0]],
 'RMSE': [lr_scores[1], rf_scores[1]],
 'MAE': [lr_scores[2], rf_scores[2]]
})
print("\nModel Comparison:")
print(results)
```

Model Comparison:

	Model	R2	RMSE	MAE
0	Linear Regression	0.017172	9.431814	5.922463
1	Random Forest Regressor	0.818989	4.047705	2.152357

```
plt.figure(figsize=(8, 5))
model_names = results['Model'].tolist()
r2_scores = results['R2'].tolist()
plt.bar(model_names, r2_scores, color=['steelblue', 'darkorange'])
plt.ylabel('R² Score')
plt.title("R² Score Comparison between Models")
plt.ylim(min(0, min(r2_scores)), 1)
plt.xticks(rotation=10)
plt.tight_layout()
plt.show()
```

R2 Score
1.0
0.8-
0.6-
0.4-
0.2-
0.0
Linear Regression
R2 Score Comparison between Models
Random Forest Regressor

EXPLAIN CODE

ChatGPT gpt-6-mini

Practical 1 – Uber Fare Prediction (Complete Code Explanation)

Your project uses **Machine Learning regression algorithms** to predict the fare of an Uber ride based on pickup location, drop-off location, distance, and pickup time.

I'll explain each step in simple language, including:

- What each line of code does.
- Why we use it.
- Important functions.
- Expected output.
- Viva questions and answers.

1. Understanding the project

Problem statement: Predict the price of an Uber ride based on its pickup and drop-off locations.

For example:

- Pickup: Location A
- Drop-off: Location B
- Distance: 5 km
- Pickup time: 8 PM

The machine learning model predicts the estimated Uber fare.

Type of Machine Learning: Supervised Learning – Regression.

Why regression? Because we are predicting a continuous numerical value (fare amount).

Step 1: Importing Libraries

Code

```
</> Python

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import (
```

```
    r2_score,
    mean_squared_error,
    mean_absolute_error
)

from math import radians, cos, sin, asin, sqrt
```

Explanation

Library	Purpose
pandas	Handles datasets and tables
numpy	Performs mathematical calculations
matplotlib	Creates graphs and visualizations
train_test_split	Splits data into training and testing
LinearRegression	Implements linear regression
RandomForestRegressor	Implements random forest regression
r2_score	Calculates R^2 score
mean_squared_error	Calculates MSE
mean_absolute_error	Calculates MAE
math	Used for distance calculations

Important viva question: Why do we import sklearn?

Answer: Scikit-learn is a Python machine learning library that provides algorithms for training, testing, and evaluating machine learning models.

Step 2: Data Loading and Preprocessing

Code

```
</> Python
df = pd.read_csv("uber.csv")

print("Initial Data Shape:", df.shape)
print(df.head())
```

Explanation:

- `pd.read_csv()` loads the Uber dataset from a CSV file.
- `df` stores the dataset in a DataFrame.
- `df.shape` displays the number of rows and columns.
- `df.head()` displays the first five records.

Your output:

```
</> Plain text

Initial Data Shape: (200000, 9)
```

This means:

- 200,000 rows.
- 9 columns.

Removing missing values

```
</> Python

df.dropna(inplace=True)
```

- `dropna()` removes rows containing missing values.
- `inplace=True` modifies the original DataFrame.

Why do we remove missing values?

Because missing values can cause errors during model training and affect prediction accuracy.

Output:

```
</> Plain text

After dropping missing values: (199999, 9)
```

One row was removed.

Converting date and time

```
</> Python
```

```
df['pickup_datetime'] = pd.to_datetime(
    df['pickup_datetime'],
    errors='coerce'
)
```

This converts the pickup datetime column into Python datetime format.

- `pd.to_datetime()` converts values into datetime.
- `errors='coerce'` converts invalid dates into NaT (Not a Time).

Extracting date and time features

```
</> Python
df['hour'] = df['pickup_datetime'].dt.hour

df['day'] = df['pickup_datetime'].dt.day

df['month'] = df['pickup_datetime'].dt.month

df['year'] = df['pickup_datetime'].dt.year

df['day_of_week'] = df['pickup_datetime'].dt.dayofweek
```

We extract useful information from the pickup datetime.

Feature	Meaning	Example
hour	Pickup hour	19
day	Day of month	31
month	Month	10
year	Year	2026
day_of_week	Day of week	5

`dayofweek` returns:

- Monday = 0
- Tuesday = 1
- Wednesday = 2
- ...
- Sunday = 6

Why do we extract these features?

Because Uber fares may depend on the time and date of the ride. For example, demand may be higher during peak hours.

Dropping unnecessary columns

```python
df.drop(
    ['pickup_datetime', 'key'],
    axis=1,
    inplace=True,
    errors='ignore'
)
```

- `drop()` removes columns.
- `axis=1` means we are removing columns, not rows.
- `inplace=True` modifies the original dataset.
- `errors='ignore'` avoids an error if a column doesn't exist.

We remove the original datetime column because we have already extracted useful features from it.

Step 3: Outlier Detection and Removal

What is an outlier?

An outlier is a value that is significantly different from normal observations.

For example:

- Normal Uber fare: ₹100–₹500 (illustrative).
- Unusually high fare: ₹50,000.

Such values may affect model performance.

Code

```python
df = df[
    (df['fare_amount'] > 0) &
    (df['fare_amount'] < 100)
]
```

This keeps only fares greater than 0 and less than 100.

Why?

- Fare cannot be negative or zero for a normal completed paid ride.
- Very high fares may be unrealistic for this dataset.

Removing invalid latitude and longitude

```python
df = df[
    (df['pickup_latitude'] <= 90) &
    (df['pickup_latitude'] >= -90)
]
```

Latitude must be between -90 and +90 degrees.

```python
df = df[
    (df['dropoff_latitude'] <= 90) &
    (df['dropoff_latitude'] >= -90)
]
```

This checks drop-off latitude.

```python
df = df[
    (df['pickup_longitude'] <= 180) &
    (df['pickup_longitude'] >= -180)
]
```

Longitude must be between -180 and +180 degrees.

```python
df = df[
    (df['dropoff_longitude'] <= 180) &
    (df['dropoff_longitude'] >= -180)
]
```

This checks drop-off longitude.

Output:

```
Plain text

Data shape after removing outliers: (199875, 12)
```

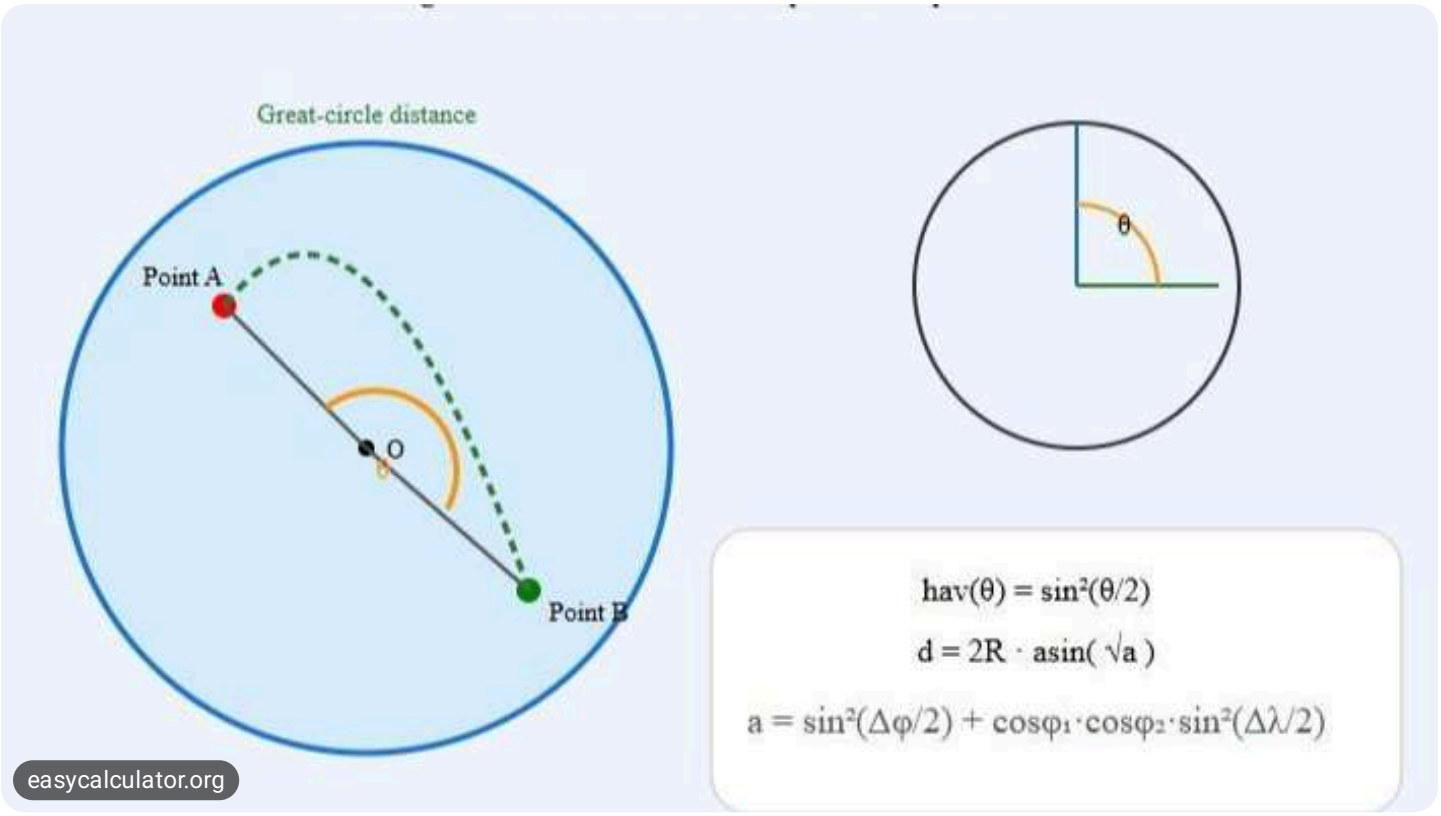
This means 124 more rows were removed.

Note: These checks remove invalid geographic values and extreme fares, but they do not guarantee that every remaining location or fare is realistic.

Step 4: Feature Engineering – Distance Calculation

This is one of the most important parts of your practical.

We calculate the distance between pickup and drop-off locations using the **Haversine Formula**.



What is the Haversine formula?

It calculates the approximate shortest distance between two points on the Earth's surface using latitude and longitude.

Code

```python
def haversine(lat1, lon1, lat2, lon2):

    lat1, lon1, lat2, lon2 = map(
        radians,
        [lat1, lon1, lat2, lon2]
    )

    dlon = lon2 - lon1
    dlat = lat2 - lat1

    a = sin(dlat/2)**2 + cos(lat1) * cos(lat2) * sin(dlon/2)**2

    c = 2 * asin(sqrt(a))

    km = 6371 * c

    return km
```

Line-by-line explanation

1. Convert degrees into radians

```python
lat1, lon1, lat2, lon2 = map(
    radians,
    [lat1, lon1, lat2, lon2]
)
```

Trigonometric functions in Python work with radians, so we convert latitude and longitude from degrees into radians.

2. Calculate longitude difference

```python
dlon = lon2 - lon1
```

Calculates the difference between two longitudes.

3. Calculate latitude difference

```python
dlat = lat2 - lat1
```

Calculates the difference between two latitudes.

4. Apply the Haversine formula

```python
a = sin(dlat/2)**2 + cos(lat1) * cos(lat2) * sin(dlon/2)**2
```

Calculates an intermediate value used in the distance formula.

5. Calculate angular distance

```python
c = 2 * asin(sqrt(a))
```

Calculates the angular distance between the two points.

6. Convert into kilometres

```python
km = 6371 * c
```

6371 is the approximate radius of Earth in kilometres.

7. Return the result

```python
return km
```

Returns the calculated distance.

Applying the function

```python
df['distance_km'] = df.apply(
    lambda x: haversine(
        x['pickup_latitude'],
        x['pickup_longitude'],
        x['dropoff_latitude'],
        x['dropoff_longitude']
    ),
    axis=1
)
```

Explanation:

- `df.apply()` applies a function to the dataset.
- `lambda x` represents one row.
- We pass pickup and drop-off coordinates to the Haversine function.
- `axis=1` means processing row by row.
- The result is stored in a new column called `distance_km`.

Removing zero-distance trips

```python
df = df[df['distance_km'] > 0]
```

Removes trips where pickup and drop-off locations are identical.

Step 5: Correlation Analysis

What is correlation?

Correlation measures the relationship between two numerical variables.

For example:

- Distance and fare.
- Passenger count and fare.
- Hour and fare.

Code

```python
correlation = df.corr(numeric_only=True)
```

Calculates correlation between numerical columns.

```python
plt.figure(figsize=(10, 6))
```

Creates a graph of width 10 and height 6.

```python
plt.imshow(
    correlation,
    cmap='coolwarm',
    aspect='auto'
)
```

Displays the correlation matrix as a heatmap.

```python
plt.colorbar(label='Correlation')
```

Displays the colour scale.

```python
plt.xticks(
    range(len(correlation.columns)),
    correlation.columns,
    rotation=45,
    ha='right'
)
```

Displays column names on the X-axis.

```python
plt.yticks(
    range(len(correlation.index)),
    correlation.index
)
```

Displays column names on the Y-axis.

```python
plt.title("Feature Correlation Heatmap")
plt.tight_layout()
plt.show()
```

Adds the title, adjusts spacing and displays the heatmap.

How to understand correlation values?

Correlation	Meaning
+1	Perfect positive correlation
0	No linear correlation
-1	Perfect negative correlation

For example, distance and fare are generally expected to have positive correlation.

Viva question: Why do we perform correlation analysis?

Answer: We perform correlation analysis to understand relationships between numerical features and identify variables that may be useful for prediction.

Step 6: Model Training

We use two machine learning algorithms:

1. Linear Regression.
2. Random Forest Regression.

A. Selecting features and target

```python
X = df[
    ['distance_km', 'hour', 'day',
     'month', 'year', 'day_of_week']
]

y = df['fare_amount']
```

- `X` contains independent variables (input features).
- `y` contains the dependent variable (output).

In our project:

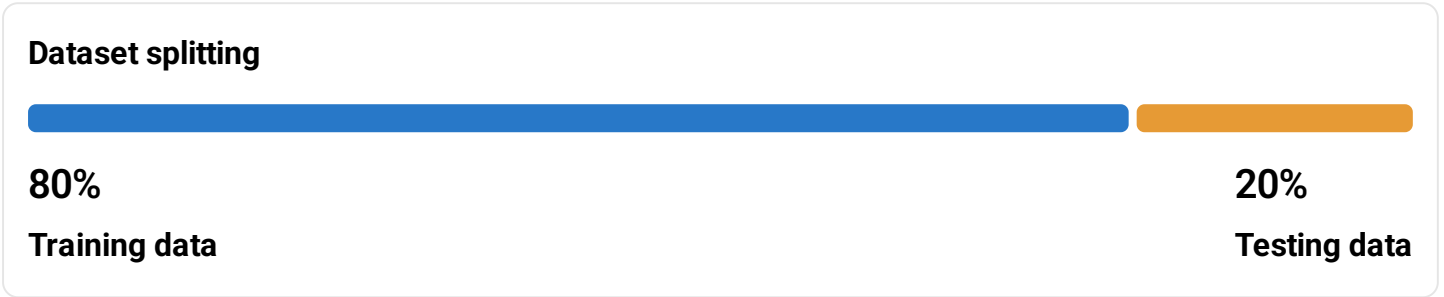
Input: Distance, hour, day, month, year and day of week.

Output: Uber fare amount.

B. Splitting the dataset

```python
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

We divide the dataset into two parts.



- Training data teaches the model.
- Testing data checks how well the model predicts unseen data.
- `random_state=42` ensures reproducible splitting.

C. Linear Regression

```python
lr_model = LinearRegression()

lr_model.fit(X_train, y_train)

y_pred_lr = lr_model.predict(X_test)
```

Explanation:

1. Create the model

```python
lr_model = LinearRegression()
```

Creates a Linear Regression model.

2. Train the model

```python
lr_model.fit(X_train, y_train)
```

The model learns the relationship between input features and fare amount.

3. Make predictions

```python
y_pred_lr = lr_model.predict(X_test)
```

Predicts fares using testing data.

What is Linear Regression?

Linear Regression is a supervised machine learning algorithm that predicts a continuous output using a linear relationship between input and output variables.

Formula:

$$y = mx + c$$

Here:

- y = Predicted fare.
- x = Input feature.
- m = Slope.
- c = Intercept.

D. Random Forest Regression

```python
rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)
```

Explanation:

1. Create Random Forest model

```python
rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
```

- `RandomForestRegressor()` creates a regression model.
- `n_estimators=100` means the forest contains 100 decision trees.
- `random_state=42` makes results reproducible.

2. Train the model

```python
rf_model.fit(X_train, y_train)
```

Trains the 100 decision trees using training data.

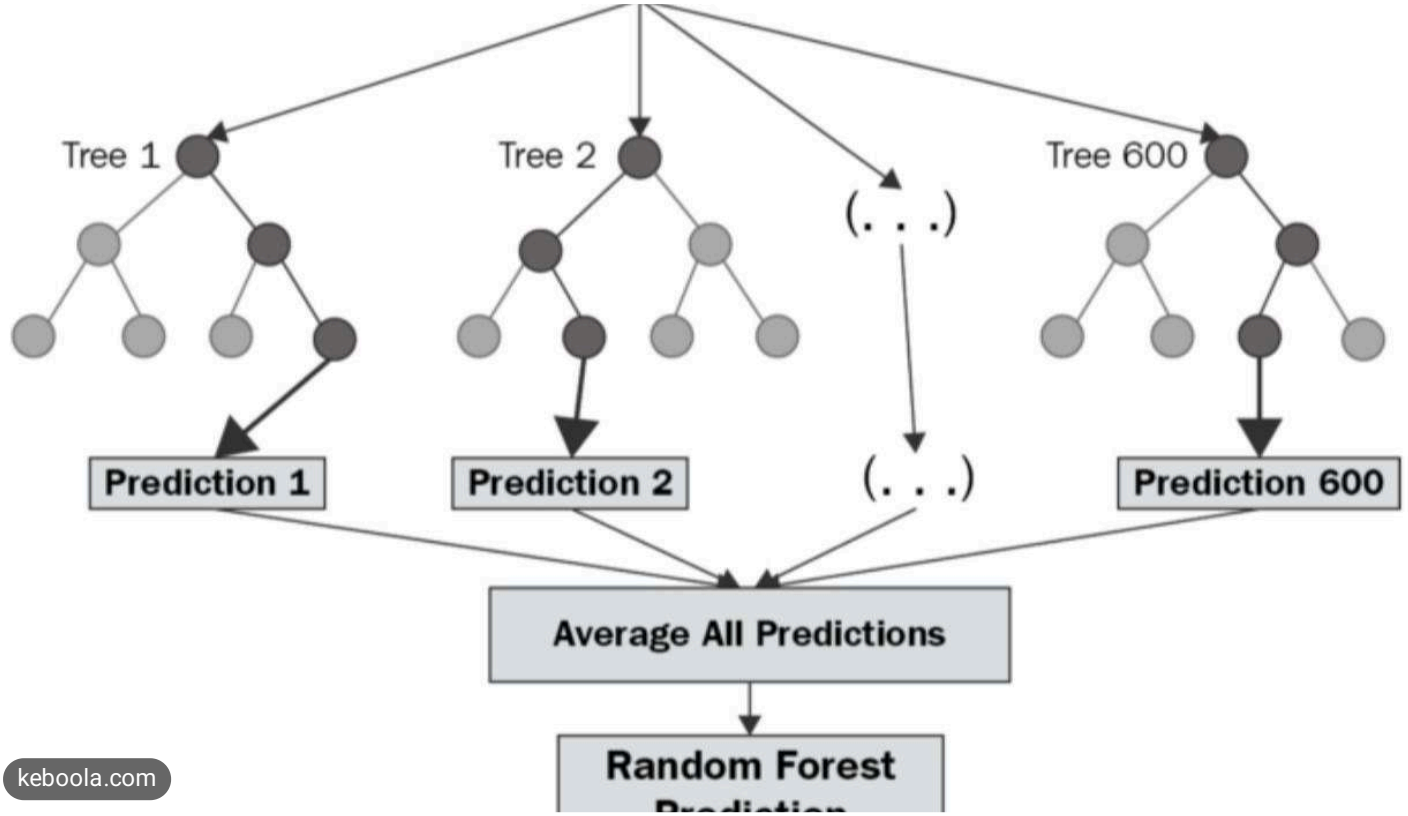
3. Make predictions

```python
y_pred_rf = rf_model.predict(X_test)
```

Predicts fare values using the trained forest.

What is Random Forest Regression?

Random Forest is an ensemble machine learning algorithm that combines predictions from multiple decision trees to produce a final prediction.



Step 7: Model Evaluation

After training, we evaluate the performance of both models.

Code

```python
def evaluate_model(y_true, y_pred, model_name):

    r2 = r2_score(y_true, y_pred)
```

```
    rmse = np.sqrt(
        mean_squared_error(y_true, y_pred)
    )

    mae = mean_absolute_error(y_true, y_pred)

    print(f"\nModel: {model_name}")
    print(f"R² Score: {r2:.4f}")
    print(f"RMSE: {rmse:.4f}")
    print(f"MAE: {mae:.4f}")

    return r2, rmse, mae
```

We create a function to calculate three evaluation metrics.

1. R² Score (Coefficient of Determination)

Measures how well the model explains variations in the actual fare values.

- 1 = Perfect prediction.
- 0 = No improvement over predicting the mean.
- Negative = Worse than predicting the mean.

Higher is better.

2. RMSE (Root Mean Squared Error)

Measures the average prediction error, with larger errors penalized more heavily.

Formula:

$$RMSE = \sqrt{\frac{\sum(y - \hat{y})^2}{n}}$$

Lower RMSE is better.

3. MAE (Mean Absolute Error)

Measures the average absolute difference between actual and predicted values.

Formula:

$$MAE = \frac{\sum |y - \hat{y}|}{n}$$

Lower MAE is better.

Evaluating both models

```python
lr_scores = evaluate_model(
    y_test,
    y_pred_lr,
    "Linear Regression"
)

rf_scores = evaluate_model(
    y_test,
    y_pred_rf,
    "Random Forest Regressor"
)
```

This calls the evaluation function for both models.

Your output

Linear Regression

0.0172

R^2 score

- RMSE: 9.4318
- MAE: 5.9225

Random Forest Regression

0.8190

R^2 score

- RMSE: 4.0477
- MAE: 2.1524

Interpretation:

Random Forest performs much better than Linear Regression because:

- It has a higher R^2 score.
- It has a lower RMSE.
- It has a lower MAE.
- It can capture more complex, nonlinear relationships.

Step 8: Model Comparison

Code

```python
results = pd.DataFrame({
    'Model': [
        'Linear Regression',
        'Random Forest Regressor'
    ],

    'R2': [
        lr_scores[0],
        rf_scores[0]
    ],

    'RMSE': [
        lr_scores[1],
        rf_scores[1]
    ],

    'MAE': [
        lr_scores[2],
        rf_scores[2]
    ]
})

print("\nModel Comparison:")
print(results)
```

Explanation:

We create a new DataFrame named `results`.

- `Model` stores model names.
- `R2` stores R^2 scores.
- `RMSE` stores RMSE values.
- `MAE` stores MAE values.

`lr_scores[0]` means Linear Regression R^2 score.

`lr_scores[1]` means Linear Regression RMSE.

`lr_scores[2]` means Linear Regression MAE.

The same applies to `rf_scores`.

Plotting the comparison graph

```python
plt.figure(figsize=(8, 5))

model_names = results['Model'].tolist()

r2_scores = results['R2'].tolist()

plt.bar(
    model_names,
    r2_scores,
    color=['steelblue', 'darkorange']
)

plt.ylabel('R² Score')

plt.title(
    "R² Score Comparison between Models"
)

plt.ylim(min(0, min(r2_scores)), 1)

plt.xticks(rotation=10)

plt.tight_layout()

plt.show()
```

Explanation:

- `plt.figure()` creates the graph.
- `tolist()` converts DataFrame columns into Python lists.
- `plt.bar()` creates a bar chart.
- `plt.ylabel()` labels the Y-axis.
- `plt.title()` adds the graph title.
- `plt.ylim()` sets the Y-axis limits.
- `plt.xticks()` rotates the X-axis labels.
- `plt.show()` displays the graph.

The Random Forest bar is much higher, indicating better performance.

Important Viva Questions and Answers

These are the questions you should prepare for your practical exam.

Question	Answer
1. What is the aim of this practical?	To predict Uber ride fares using regression algorithms.
2. What is regression?	A supervised learning technique used to predict continuous numerical values.
3. Which dataset is used?	Uber fare dataset (`uber.csv`).
4. What is preprocessing?	Cleaning and transforming raw data before model training.
5. What is an outlier?	A value that differs significantly from normal observations.
6. What is correlation?	It measures the relationship between two numerical variables.
7. What is the Haversine formula?	A formula used to calculate geographical distance between two points on Earth.
8. Why use Linear Regression?	To predict continuous values using a linear relationship.
9. Why use Random Forest?	It combines multiple decision trees to improve prediction performance.
10. What is `n_estimators=100`?	It specifies 100 decision trees in the Random Forest.
11. Why split the dataset?	To train the model and test its performance on unseen data.
12. What is R^2?	It measures how well the model explains variations in the target variable.
13. What is RMSE?	Root Mean Squared Error; it measures prediction errors.
14. What is MAE?	Mean Absolute Error; it measures average absolute prediction errors.
15. Which model performs better?	Random Forest Regression.

Important concepts to remember

Remember these 8 things for your practical:

1. `pd.read_csv()` – Loads dataset.
2. `dropna()` – Removes missing values.
3. `pd.to_datetime()` – Converts date and time.
4. `haversine()` – Calculates geographical distance.
5. `corr()` – Calculates correlation.
6. `train_test_split()` – Splits training and testing data.
7. `.fit()` – Trains the model.
8. `.predict()` – Predicts the output.

Final conclusion for your practical file

You can write this in your journal:

The Uber fare prediction system was implemented using Linear Regression and Random Forest Regression algorithms. The dataset was preprocessed by removing missing values and outliers. The Haversine formula was used to calculate the geographical distance between pickup and drop-off locations. Correlation analysis was performed to understand relationships between features. Both models were trained and evaluated using R^2 score, RMSE, and MAE. Random Forest Regression achieved an R^2 score of 0.8190, while Linear Regression achieved 0.0172. Therefore, Random Forest Regression performed better on the given test dataset.

My suggestion: For your practical exam, focus especially on the Haversine formula, difference between Linear Regression and Random Forest, `fit()` vs `predict()`, and R^2/RMSE/MAE. These are important concepts the examiner can ask you to explain or modify in the code.

2026-10-31 16:14:35 **You**

Step 1 : Import Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
Step 2 : Load dataset
df = pd.read_csv("emails.csv", encoding="ISO-8859-1") # Adjust path
if needed
Drop unnecessary columns if present
if "Email No." in df.columns:
 df = df.drop(columns=["Email No."])
Ensure label is integer
df["Prediction"] = df["Prediction"].astype(int)
Features & target
X = df.drop(columns=["Prediction"])
y = df["Prediction"]
Print basic info
print(df.columns)
print(df.head(5))
Index(['the', 'to', 'ect', 'and', 'for', 'of', 'a', 'you', 'hou',

```
'in',
 ...
 'connevey', 'jay', 'valued', 'lay', 'infrastructure',
'military',
 'allowing', 'ff', 'dry', 'Prediction'],
 dtype='object', length=3001)
 the to ect and for of a you hou in ... connevey jay
valued \
0 0 0 1 0 0 0 2 0 0 0 ... 0 0
0
1 8 13 24 6 6 2 102 1 27 18 ... 0 0
0
2 0 0 1 0 0 0 8 0 0 4 ... 0 0
0
3 0 5 22 0 5 1 51 2 10 1 ... 0 0
0
4 7 6 17 1 5 2 57 0 9 3 ... 0 0
0
 lay infrastructure military allowing ff dry Prediction
0 0 0 0 0 0 0 0
1 0 0 0 0 1 0 0
2 0 0 0 0 0 0 0
3 0 0 0 0 0 0 0
4 0 0 0 0 1 0 0
[5 rows x 3001 columns]
Step 3 : Data Splitting (training & testing)
X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42, stratify=y
)
Step 4 : KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
print("\n--- KNN Performance ---")
print("Accuracy:", accuracy_score(y_test, y_pred_knn))
print("Classification Report:\n", classification_report(y_test,
y_pred_knn))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn))
--- KNN Performance ---
Accuracy: 0.8714975845410629
Classification Report:
```

```
 precision recall f1-score support
 0 0.93 0.88 0.91 735
 1 0.75 0.84 0.79 300
 accuracy 0.87 1035
 macro avg 0.84 0.86 0.85 1035
weighted avg 0.88 0.87 0.87 1035
Confusion Matrix:
 [[649 86]
 [ 47 253]]
step 5 : SVM
svm = SVC(kernel='linear', random_state=42)
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
print("\n--- SVM Performance ---")
print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print("Classification Report:\n", classification_report(y_test,
y_pred_svm))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svm))
--- SVM Performance ---
Accuracy: 0.9671497584541063
Classification Report:
 precision recall f1-score support
 0 0.98 0.97 0.98 735
 1 0.94 0.95 0.94 300
 accuracy 0.97 1035
 macro avg 0.96 0.96 0.96 1035
weighted avg 0.97 0.97 0.97 1035
Confusion Matrix:
 [[716 19]
 [ 15 285]]
Step 6 : Plotting
def plot_confusion_matrix(matrix, axis, title, cmap):
 image = axis.imshow(matrix, cmap=cmap)
 axis.set_title(title)
 axis.set_xlabel("Predicted")
 axis.set_ylabel("Actual")
 axis.set_xticks([0, 1])
 axis.set_yticks([0, 1])
 for row in range(matrix.shape[0]):
 for column in range(matrix.shape[1]):
 axis.text(column, row, matrix[row, column], ha="center",
```

```
va="center")
 return image
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
plot_confusion_matrix(
 confusion_matrix(y_test, y_pred_knn), ax[0], "KNN Confusion
Matrix", "Blues"
)
plot_confusion_matrix(
 confusion_matrix(y_test, y_pred_svm), ax[1], "SVM Confusion
Matrix", "Greens"
)
plt.tight_layout()
plt.show()
Actual
KNN Confusion Matrix
0- 649 86 0| 716 19 Actual
SVM Confusion Matrix
1- 47 253 1- 15 285
0| 1
Predicted
0 1
Predicted
```

ChatGPT gpt-6-mini

Practical – Email Spam Detection using KNN and SVM

Your practical is about **classifying emails as Spam or Not Spam (Ham)** using two Machine Learning algorithms:

1. K-Nearest Neighbors (KNN)
2. Support Vector Machine (SVM)

I'll explain every step in simple language, including the code, output, important functions, and viva questions.

1. Understanding the project

Aim: To classify emails into two categories:

- `0` = Not Spam (Ham)
- `1` = Spam

Example:

Email	Prediction
Congratulations! You won a lottery.	Spam (1)
Please attend tomorrow's meeting.	Not Spam (0)
Claim your free cash prize now!	Spam (1)

Type of Machine Learning: Supervised Learning – Classification.

Why classification? Because we are predicting categories (Spam or Not Spam), not continuous numerical values.

Step 1: Import Libraries

Code

```python
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

import matplotlib.pyplot as plt
```

Explanation

Library / Function	Purpose
pandas	Loads and handles datasets
train_test_split	Divides dataset into training and testing
KNeighborsClassifier	Implements KNN algorithm

SVC	Implements Support Vector Machine
accuracy_score	Calculates prediction accuracy
classification_report	Displays precision, recall and F1-score
confusion_matrix	Shows correct and incorrect predictions
matplotlib.pyplot	Creates graphs

Viva: What is Scikit-learn?

Answer: Scikit-learn is a Python library used for machine learning. It provides algorithms for classification, regression, clustering and model evaluation.

Step 2: Load Dataset

Code

```python
df = pd.read_csv(
    "emails.csv",
    encoding="ISO-8859-1"
)
```

Explanation:

- `pd.read_csv()` loads the CSV dataset.
- `"emails.csv"` is the dataset filename.
- `encoding="ISO-8859-1"` specifies the character encoding used to read the file.

Removing unnecessary column

```python
if "Email No." in df.columns:
    df = df.drop(columns=["Email No."])
```

Explanation:

- `if` checks whether the column exists.
- `df.columns` displays all column names.
- `df.drop()` removes the unnecessary column.

- `columns=["Email No."]` specifies which column to remove.

Why remove it?

Because Email No. is just an identifier and does not provide useful information for spam classification.

Converting prediction into integer

```python
df["Prediction"] = df["Prediction"].astype(int)
```

This converts the Prediction column into integer format.

- `0` means Not Spam.
- `1` means Spam.

`astype(int)` converts the column into integer data type.

Selecting features and target

```python
X = df.drop(columns=["Prediction"])

y = df["Prediction"]
```

This is a very important part.

- `X` = Input features.
- `y` = Output or target variable.

Your dataset contains 3001 columns initially. After removing the Email No. column, there are 3000 word-frequency features and one target column.

For example:

the	to	ect	and	for	Prediction
0	0	1	0	0	0
8	13	24	6	6	0
0	0	1	0	0	0

These numerical values represent how frequently particular words appear in an email.

Displaying dataset information

```python
print(df.columns)

print(df.head(5))
```

- `df.columns` displays column names.
- `df.head(5)` displays the first five records.

Viva: What is a feature?

Answer: A feature is an input variable used by a machine learning model to make predictions.

Step 3: Data Splitting (Training and Testing)

Code

```python
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Explanation

We divide our dataset into two parts.

Dataset splitting



80%
Training data

20%
Testing data

- `X_train` : Input training data.

- `X_test`: Input testing data.
- `y_train`: Actual output for training data.
- `y_test`: Actual output for testing data.

What is `test_size=0.2`?

It means 20% of the dataset is used for testing and 80% for training.

What is `random_state=42`?

It ensures that the same random split is produced whenever we run the code again.

What is `stratify=y`?

It maintains approximately the same proportion of Spam and Not Spam emails in both training and testing datasets.

Your testing dataset contains 1035 records:

- 735 Not Spam.
- 300 Spam.

Step 4: KNN Algorithm

Algorithm 1 K-Nearest Neighbors

What is KNN?

KNN stands for K-Nearest Neighbors.

It classifies a new data point based on the majority class of its nearest neighboring data points.

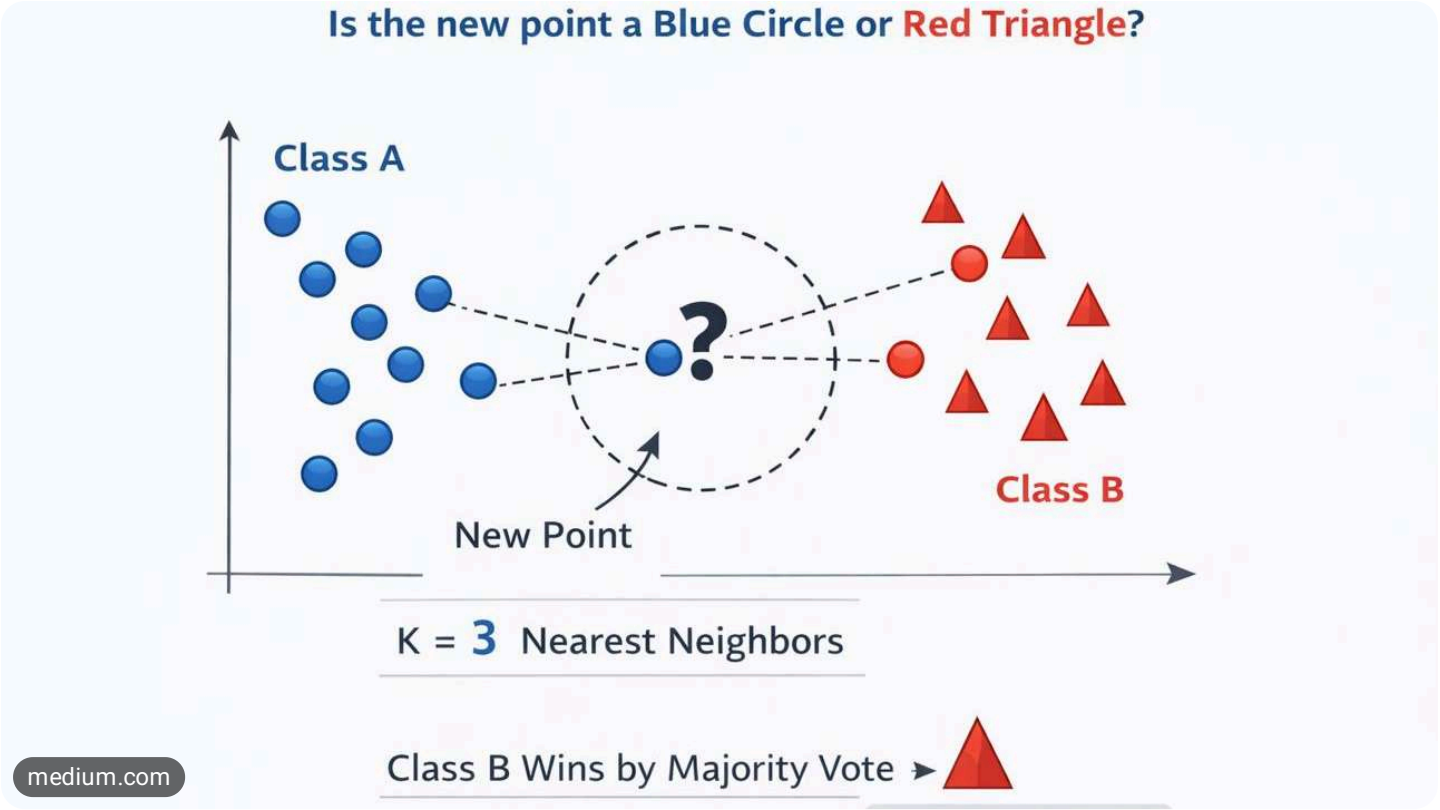
For example:

Suppose we choose K = 5.

Out of the five nearest emails:

- 4 are Spam.
- 1 is Not Spam.

The new email is classified as Spam.



Code

```python
knn = KNeighborsClassifier(n_neighbors=5)
```

Creates a KNN classifier.

`n_neighbors=5` means the algorithm checks the five nearest neighbors before deciding the class.

Training the model

```python
knn.fit(X_train, y_train)
```

- `.fit()` trains the model.
- It learns from the training dataset.

Making predictions

```python
y_pred_knn = knn.predict(X_test)
```

- `.predict()` predicts whether each testing email is Spam or Not Spam.
- Results are stored in `y_pred_knn`.

Evaluating KNN

```python
print("\n--- KNN Performance ---")

print("Accuracy:", accuracy_score(y_test, y_pred_knn))

print(
    "Classification Report:\n",
    classification_report(y_test, y_pred_knn)
)

print(
    "Confusion Matrix:\n",
    confusion_matrix(y_test, y_pred_knn)
)
```

This displays the accuracy, classification report and confusion matrix.

Your KNN output

87.15%

KNN Accuracy Confusion matrix:

Actual / Predicted	0	1
Not Spam (0)	649	86
Spam (1)	47	253

Interpretation:

- 649 Not Spam emails correctly classified.
- 253 Spam emails correctly classified.
- 86 Not Spam emails incorrectly classified as Spam.
- 47 Spam emails incorrectly classified as Not Spam.

Step 5: SVM Algorithm

Algorithm 2 Support Vector Machine

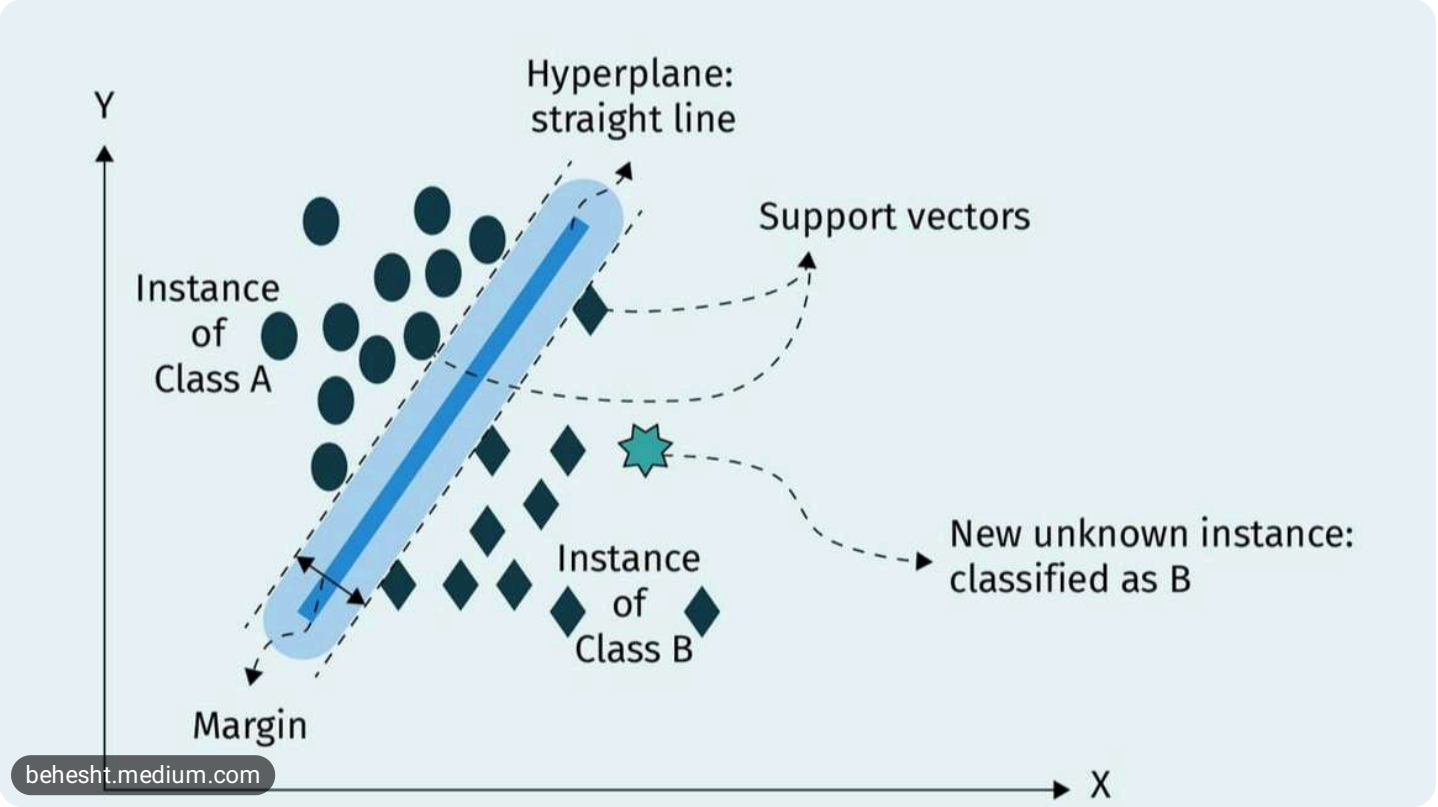
What is SVM?

SVM stands for Support Vector Machine.

It is a supervised machine learning algorithm used for classification.

It finds the best decision boundary (hyperplane) that separates different classes.

For example, it separates Spam emails from Not Spam emails.



Code

```python
svm = SVC(
    kernel='linear',
    random_state=42
)
```

Explanation:

- `SVC()` creates a Support Vector Classifier.
- `kernel='linear'` uses a linear decision boundary.
- `random_state=42` sets the random seed.

Training the model

```python
svm.fit(X_train, y_train)
```

Trains the SVM model using training data.

Making predictions

```python
y_pred_svm = svm.predict(X_test)
```

Predicts whether the testing emails are Spam or Not Spam.

Evaluating SVM

```python
print("\n--- SVM Performance ---")

print("Accuracy:", accuracy_score(y_test, y_pred_svm))

print(
    "Classification Report:\n",
    classification_report(y_test, y_pred_svm)
)

print(
    "Confusion Matrix:\n",
    confusion_matrix(y_test, y_pred_svm)
)
```

Your SVM output

96.71%

SVM Accuracy Confusion matrix:

Actual / Predicted	0	1

Not Spam (0)	716	19
Spam (1)	15	285

Interpretation:

- 716 Not Spam emails correctly classified.
- 285 Spam emails correctly classified.
- 19 Not Spam emails incorrectly classified as Spam.
- 15 Spam emails incorrectly classified as Not Spam.

Important: Understanding Classification Metrics

Your examiner may ask you to explain these four terms.

1. Accuracy

Accuracy measures the percentage of total correct predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Higher accuracy is better.

2. Precision

Precision tells us how many emails predicted as Spam were actually Spam.

$$Precision = \frac{TP}{TP + FP}$$

Higher precision means fewer genuine emails are wrongly marked as Spam.

3. Recall

Recall tells us how many actual Spam emails were correctly identified.

$$Recall = \frac{TP}{TP + FN}$$

Higher recall means fewer Spam emails are missed.

4. F1-score

F1-score is the harmonic mean of precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

It balances precision and recall.

Step 6: Confusion Matrix Plotting

A confusion matrix visually represents correct and incorrect predictions.

Code – Creating the function

```python
def plot_confusion_matrix(matrix, axis, title, cmap):

    image = axis.imshow(matrix, cmap=cmap)

    axis.set_title(title)

    axis.set_xlabel("Predicted")

    axis.set_ylabel("Actual")

    axis.set_xticks([0, 1])

    axis.set_yticks([0, 1])

    for row in range(matrix.shape[0]):
        for column in range(matrix.shape[1]):

            axis.text(
                column,
                row,
                matrix[row, column],
                ha="center",
                va="center"
            )

    return image
```

Explanation

1. Define function

```python
def plot_confusion_matrix(matrix, axis, title, cmap):
```

Creates a function to plot the confusion matrix.

It accepts four arguments:

- `matrix` : Confusion matrix values.
- `axis` : Graph area.
- `title` : Graph title.
- `cmap` : Colour map.

2. Display matrix

```python
image = axis.imshow(matrix, cmap=cmap)
```

Displays the confusion matrix using colours.

3. Add labels

```python
axis.set_title(title)
axis.set_xlabel("Predicted")
axis.set_ylabel("Actual")
```

Adds the title and axis labels.

4. Set X-axis and Y-axis values

```python
axis.set_xticks([0, 1])
axis.set_yticks([0, 1])
```

Displays two classes:

- 0 = Not Spam.
- 1 = Spam.

5. Display numerical values

```python
for row in range(matrix.shape[0]):
    for column in range(matrix.shape[1]):

        axis.text(
            column,
            row,
            matrix[row, column],
            ha="center",
            va="center"
        )
```

- Outer loop moves through rows.
- Inner loop moves through columns.
- `axis.text()` displays the numerical values inside the graph.
- `ha="center"` aligns text horizontally.
- `va="center"` aligns text vertically.

6. Return image

```python
return image
```

Returns the plotted matrix image.

Creating two graphs

```python
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
```

Creates one figure containing two graphs.

- `1` = One row.
- `2` = Two columns.
- `figsize=(12,5)` = Graph size.

Plotting KNN confusion matrix

```python
plot_confusion_matrix(
    confusion_matrix(y_test, y_pred_knn),
    ax[0],
    "KNN Confusion Matrix",
    "Blues"
)
```

Displays the KNN confusion matrix in the first graph.

Plotting SVM confusion matrix

```python
plot_confusion_matrix(
    confusion_matrix(y_test, y_pred_svm),
    ax[1],
    "SVM Confusion Matrix",
    "Greens"
)
```

Displays the SVM confusion matrix in the second graph.

Final commands

```python
plt.tight_layout()
plt.show()
```

- `tight_layout()` adjusts graph spacing.
- `show()` displays both graphs.

Final Comparison: KNN vs SVM

Model accuracy comparison



Better performing model

SVM SVM achieved approximately 9.57 percentage points higher accuracy than KNN.

Metric	KNN	SVM
Accuracy	87.15%	96.71%
Spam Precision	75%	94%
Spam Recall	84%	95%
Spam F1-score	79%	94%

Conclusion: SVM performs better than KNN on this dataset because it achieves higher accuracy, precision, recall and F1-score.

Important Viva Questions

Here are 20 questions you should prepare.

No.	Question	Answer
1	What is the aim of this practical?	To detect Spam emails using KNN and SVM.
2	What is Spam?	Unwanted or unsolicited email messages.
3	What is Ham?	A genuine or non-spam email.
4	What type of ML is used?	Supervised learning.
5	What type of problem is this?	Binary classification.
6	What is KNN?	K-Nearest Neighbors algorithm.
7	What does K represent?	Number of nearest neighbors.
8	What is SVM?	Support Vector Machine.
9	What is a hyperplane?	A decision boundary separating classes.
10	What is a kernel?	A function used to determine the decision boundary in SVM.
11	What is a linear kernel?	A kernel that creates a linear decision boundary.
12	Why do we use train_test_split?	To divide data into training and testing sets.
13	What is accuracy?	Percentage of correct predictions.
14	What is precision?	Correct positive predictions divided by all positive predictions.
15	What is recall?	Correct positive predictions divided by all actual positives.

16	What is F1-score?	Harmonic mean of precision and recall.
17	What is a confusion matrix?	A table showing correct and incorrect classifications.
18	What does `.fit()` do?	Trains the model.
19	What does `.predict()` do?	Predicts output for new data.
20	Which algorithm performed better?	SVM, with 96.71% accuracy.

5 important differences to remember

KNN	SVM
Instance-based algorithm	Margin-based algorithm
Uses nearest neighbors	Uses a decision boundary
Uses majority voting	Uses support vectors
K value is important	Kernel choice is important
Can be slower during prediction	Can be computationally expensive during training

Final conclusion for your practical journal

You can write:

The Email Spam Detection system was successfully implemented using K-Nearest Neighbors (KNN) and Support Vector Machine (SVM) algorithms. The email dataset was loaded and divided into training and testing datasets. Both algorithms were trained to classify emails as Spam or Not Spam. Their performance was evaluated using accuracy, precision, recall, F1-score and confusion matrix. KNN achieved an accuracy of 87.15%, whereas SVM achieved an accuracy of 96.71%. Therefore, SVM performed better than KNN for the given dataset.

Exam preparation tip

Focus on these five topics:

1. Difference between classification and regression.
2. Difference between KNN and SVM.
3. Meaning of `n_neighbors=5`.
4. Confusion matrix (TP, TN, FP, FN).
5. Accuracy, precision, recall and F1-score.

These will help you explain the practical even if the examiner asks you to modify the code.

Practical-3: Gradient Descent Algorithm

Problem statement: Implement the gradient descent algorithm to find the local minimum of

y=(x +3)

2

, starting from x=2.

Steps

1. Define the function and its derivative
2. Initialize parameters
3. Run the gradient descent loop
4. Print the result
5. Plot convergence

Step 1: Import Libraries

Step 2: Define the function and its derivative

Step 3: Initialize parameters for gradient descent

Step 4: Gradient descent loop

```
import numpy as np
def f(x):
return (x + 3)**2
def grad_f(x):
return 2 * (x + 3)
x_current = 2 # starting point
learning_rate = 0.1 # step size
tolerance = 1e-6 # convergence tolerance
max_iterations = 25 # maximum iterations
history = [x_current] # sotring history
for i in range(max_iterations):
gradient = grad_f(x_current)
x_next = x_current - learning_rate * gradient # update step
# Check convergence
if abs(x_next - x_current) < tolerance:
print(f"Converged after {i+1} iterations.")
break
x_current = x_next
history.append(x_current)
print(f"Iteration {i+1}: x = {x_current:.4f}, f(x) =
{f(x_current):.4f}")
```

```
Iteration 1: x = 1.0000, f(x) = 16.0000
Iteration 2: x = 0.2000, f(x) = 10.2400
Iteration 3: x = -0.4400, f(x) = 6.5536
```

Iteration 4: x = -0.9520, f(x) = 4.1943
Iteration 5: x = -1.3616, f(x) = 2.6844
Iteration 6: x = -1.6893, f(x) = 1.7180
Iteration 7: x = -1.9514, f(x) = 1.0995
Iteration 8: x = -2.1611, f(x) = 0.7037
Iteration 9: x = -2.3289, f(x) = 0.4504
Iteration 10: x = -2.4631, f(x) = 0.2882
Iteration 11: x = -2.5705, f(x) = 0.1845
Iteration 12: x = -2.6564, f(x) = 0.1181
Iteration 13: x = -2.7251, f(x) = 0.0756
Iteration 14: x = -2.7801, f(x) = 0.0484
Iteration 15: x = -2.8241, f(x) = 0.0309
Iteration 16: x = -2.8593, f(x) = 0.0198
Iteration 17: x = -2.8874, f(x) = 0.0127
Iteration 18: x = -2.9099, f(x) = 0.0081
Iteration 19: x = -2.9279, f(x) = 0.0052
Iteration 20: x = -2.9424, f(x) = 0.0033
Iteration 21: x = -2.9539, f(x) = 0.0021
Iteration 22: x = -2.9631, f(x) = 0.0014
Iteration 23: x = -2.9705, f(x) = 0.0009
Iteration 24: x = -2.9764, f(x) = 0.0006
Iteration 25: x = -2.9811, f(x) = 0.0004
Step 5: Print the result
Step 6: Plotting
print("Local minima at x =", x_current)
print("Function value at local minima y =", f(x_current))
Local minima at x = -2.981110534068521
Function value at local minima y = 0.00035681192317650156
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.plot(history, [f(val) for val in history], marker='o')
plt.xlabel("x values")
plt.ylabel("f(x)")
plt.title("Gradient Descent Convergence")
plt.grid()
plt.savefig("code_3_convergence.png")
print("Plot saved to code_3_convergence.png")
Plot saved to code_3_convergence.png

Practical 3 – Gradient Descent Algorithm

Your practical implements the **Gradient Descent Algorithm** to find the local minimum of the mathematical function:

$$y = (x + 3)^2$$

Starting point:

$$x = 2$$

I'll explain the complete code line by line, mathematical calculations, output, graph and important viva questions in simple language.

1. Understanding the practical

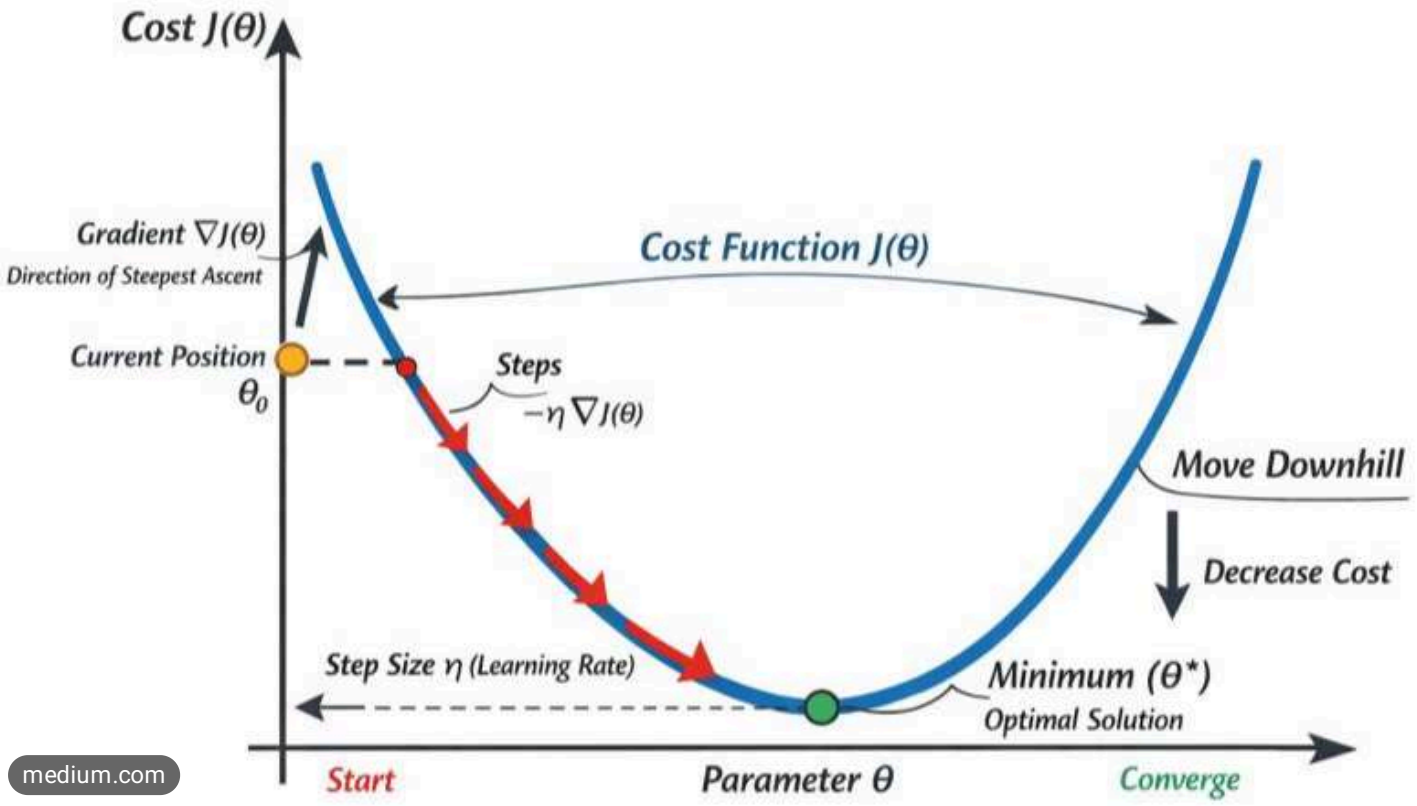
Aim: To implement the Gradient Descent Algorithm to find the minimum value of a given mathematical function.

What is Gradient Descent?

Gradient Descent is an optimization algorithm used to find the minimum value of a function by repeatedly updating the input value in the direction opposite to its gradient (derivative).

In simple words:

Imagine standing on a hill and trying to reach the lowest point. You take small steps downhill until you reach the bottom.



In your practical:

- Function: $y = (x + 3)^2$
- Starting point: $x = 2$
- Expected minimum: $x = -3$
- Minimum function value: $y = 0$

Type: Optimization algorithm.

Step 1: Import Libraries

Code

```python
import numpy as np
```

Explanation

- `numpy` is a Python library used for mathematical and numerical calculations.
- `np` is an alias (short name) for NumPy.

Why do we use NumPy?

It provides functions for mathematical calculations and numerical operations.

Note: Your current implementation doesn't actually use any NumPy function, so this import is optional.

Step 2: Define the Function and Its Derivative

Code

```python
def f(x):
    return (x + 3)**2
```

Explanation

- `def` is used to define a function.
- `f(x)` is the name of our function.
- `x` is the input value.
- `return` gives the calculated output.
- `**2` means square.

Our mathematical function is:

$$f(x) = (x + 3)^2$$

For example, if $x = 2$:

$$f(2) = (2 + 3)^2$$

$$= 5^2 = 25$$

Therefore, the output is 25.

Defining the derivative

```python
def grad_f(x):
    return 2 * (x + 3)
```

This function calculates the derivative (gradient) of the original function.

Our original function:

$$f(x) = (x + 3)^2$$

Differentiating using the power rule:

$$f'(x) = 2(x + 3)$$

Therefore:

```python
return 2 * (x + 3)
```

calculates the derivative.

Example: When $x = 2$:

$$f'(2) = 2(2 + 3) = 10$$

The gradient is 10.

Important viva question: What is a derivative?

Answer: A derivative represents the rate of change of a function with respect to its input variable. In gradient descent, it tells us the direction and slope of the function.

Step 3: Initialize Parameters for Gradient Descent

Code

```python
x_current = 2
learning_rate = 0.1
tolerance = 1e-6
max_iterations = 25
history = [x_current]
```

Let's understand each variable.

1. Starting point

```python
x_current = 2
```

We start the algorithm from x = 2.

2. Learning rate

```python
learning_rate = 0.1
```

Learning rate controls how big a step we take during each iteration.

- Large learning rate = Bigger steps.
- Small learning rate = Smaller steps.

Here, the learning rate is 0.1.

3. Tolerance

```python
tolerance = 1e-6
```

Tolerance defines how small the change in x must become before the algorithm stops.

`1e-6` means:

$$0.000001$$

4. Maximum iterations

```python
max_iterations = 25
```

The algorithm will perform a maximum of 25 iterations.

5. History

```python
history = [x_current]
```

Stores the starting x value and subsequent values during gradient descent.

Why?

Because we use these values later to plot the convergence graph.

Step 4: Gradient Descent Loop

This is the most important part of your practical.

Code

```python
for i in range(max_iterations):

    gradient = grad_f(x_current)

    x_next = x_current - learning_rate * gradient

    if abs(x_next - x_current) < tolerance:
        print(f"Converged after {i+1} iterations.")
        break

    x_current = x_next

    history.append(x_current)

    print(
        f"Iteration {i+1}: x = {x_current:.4f}, "
        f"f(x) = {f(x_current):.4f}"
    )
```

Line-by-line explanation

1. Start the loop

```python
```

```
for i in range(max_iterations):
```

Runs the loop up to 25 times.

`i` represents the iteration number, starting from 0.

2. Calculate gradient

```
</> Python

gradient = grad_f(x_current)
```

Calls the derivative function and calculates the gradient at the current x value.

For the first iteration:

$$gradient = 2(2 + 3) = 10$$

3. Update x value

```
</> Python

x_next = x_current - learning_rate * gradient
```

This is the most important formula in Gradient Descent.

$$\boxed{x_{new} = x_{old} - \alpha f'(x)}$$

Where:

- x_{new} = New x value.
- x_{old} = Previous x value.
- α = Learning rate.
- $f'(x)$ = Gradient.

Let's calculate the first iteration.

Given:

- Current x = 2
- Learning rate = 0.1
- Gradient = 10

Substitute:

$$x_{new} = 2 - (0.1 \times 10)$$

$$x_{new} = 2 - 1$$

$$\boxed{x_{new} = 1}$$

This matches your output.

4. Check convergence

```python
if abs(x_next - x_current) < tolerance:
```

Checks whether the difference between the old and new x values is smaller than the tolerance.

- `abs()` calculates absolute value.
- If the difference is sufficiently small, the algorithm stops.

5. Break the loop

```python
print(f"Converged after {i+1} iterations.")
break
```

Displays the convergence message and terminates the loop.

6. Update current x

```python
x_current = x_next
```

Replaces the old x value with the new x value.

7. Store history

```python
```

```
history.append(x_current)
```

Stores the updated x value in the history list.

8. Print iteration results

```python
print(
    f"Iteration {i+1}: x = {x_current:.4f}, "
    f"f(x) = {f(x_current):.4f}"
)
```

Displays:

- Iteration number.
- Current x value.
- Function value.

`.4f` means display the number up to four decimal places.

---

2. Understand the first five iterations

Your examiner might ask you to calculate iterations manually.

Iteration	Old x	Gradient	New x
1	2.0000	10.0000	1.0000
2	1.0000	8.0000	0.2000
3	0.2000	6.4000	-0.4400
4	-0.4400	5.1200	-0.9520
5	-0.9520	4.0960	-1.3616

Notice something?

The x value gradually moves towards -3.

This is how Gradient Descent finds the minimum.

Step 5: Print the Result

Code

```python
print("Local minima at x =", x_current)

print(
    "Function value at local minima y =",
    f(x_current)
)
```

Explanation

First statement:

Displays the final x value.

Second statement:

Calculates the function value using the final x.

Your output:

Final result

Local minimum x

-2.9811105

Function value y

0.0003568

The theoretical answer is:

$$x = -3$$

$$y = (-3 + 3)^2 = 0$$

Your result is very close to the theoretical minimum.

Why isn't it exactly -3?

Because you have limited the algorithm to 25 iterations.

Step 6: Plotting – Gradient Descent Convergence

Code

```python
import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
```

Explanation:

- `matplotlib` is a visualization library.
- `"Agg"` selects a non-interactive plotting backend.
- `pyplot` is used to create graphs.

Plotting the graph

```python
plt.plot(
    history,
    [f(val) for val in history],
    marker='o'
)
```

Explanation:

- `history` contains all x values.
- `[f(val) for val in history]` calculates the function value for every x.
- `marker='o'` displays circular markers for each point.

```python
plt.xlabel("x values")
```

Sets the X-axis label.

```python
```

```
plt.ylabel("f(x)")
```

Sets the Y-axis label.

```
</> Python
plt.title("Gradient Descent Convergence")
```

Sets the graph title.

```
</> Python
plt.grid()
```

Displays grid lines.

```
</> Python
plt.savefig("code_3_convergence.png")
```

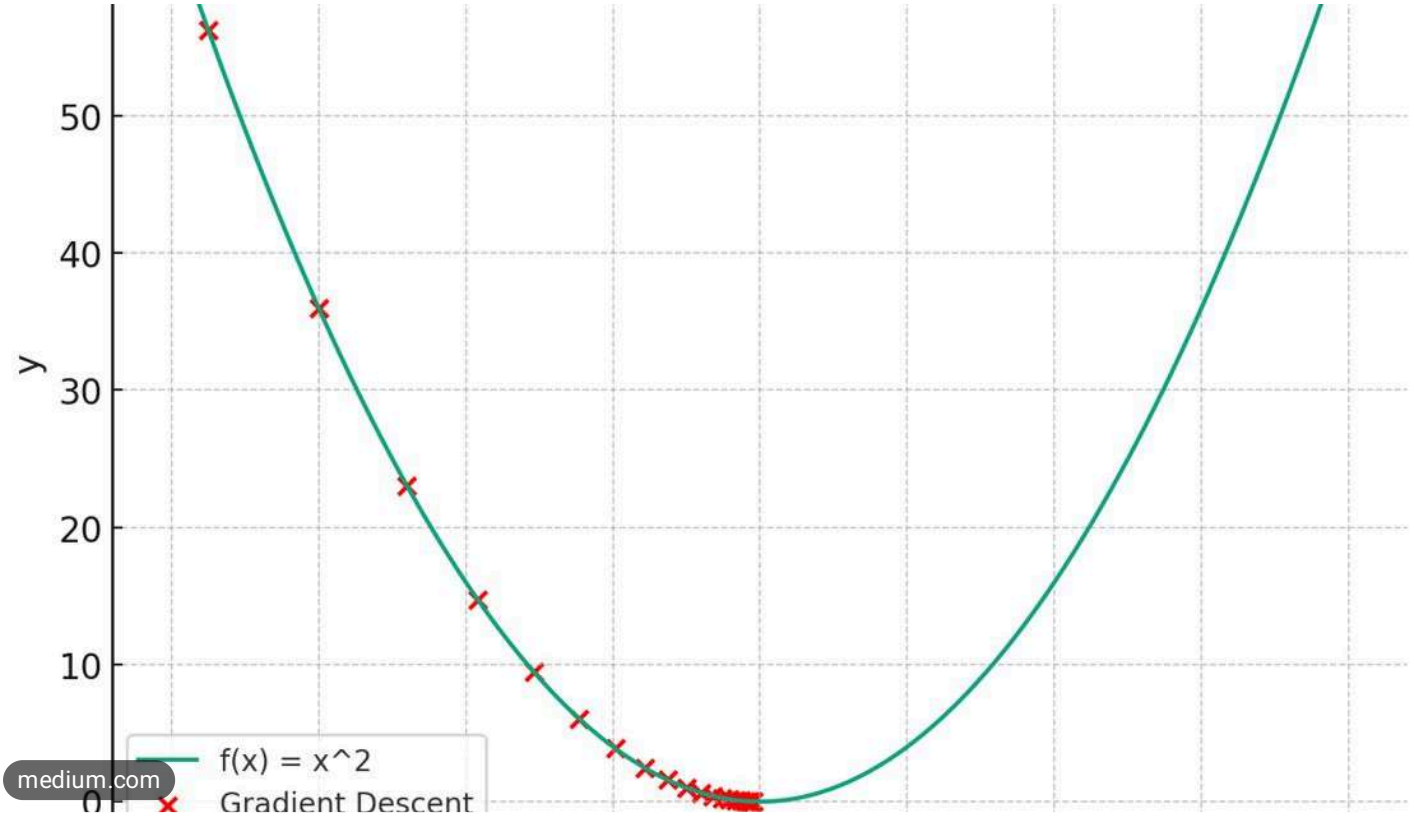
Saves the graph as an image named `code_3_convergence.png`.

```
</> Python
print("Plot saved to code_3_convergence.png")
```

Displays the confirmation message.

What is convergence?

Convergence means the algorithm is approaching a stable solution or minimum value.



The graph shows how the function value decreases as the algorithm approaches the minimum.

Important Viva Questions and Answers

No.	Question	Answer
1	What is Gradient Descent?	An optimization algorithm used to find the minimum of a function.
2	What is the aim of this practical?	To find the local minimum of a given function using Gradient Descent.
3	Which function is used?	$y = (x + 3)^2$.
4	What is the starting point?	x = 2.
5	What is the derivative of the function?	$2(x + 3)$.
6	What is the learning rate?	It controls the size of each update step.
7	What is the learning rate in this practical?	0.1.
8	What is tolerance?	The minimum change allowed before stopping the algorithm.
9	What is an iteration?	One complete update step in the algorithm.
10	What is convergence?	When the algorithm approaches a stable solution.
11	What does `abs()` do?	Returns the absolute value.
12	What does `break` do?	Terminates the loop.
13	What does `append()` do?	Adds an element to a list.

14	What is `max_iterations`?	Maximum number of iterations allowed.
15	What is the final minimum value of x?	Theoretically, -3.
16	What is the minimum value of y?	0.
17	Why do we use a derivative?	To determine the direction and slope of the function.
18	What happens if the learning rate is too high?	The algorithm may overshoot or diverge.
19	What happens if the learning rate is too low?	Convergence becomes slow.
20	Where is Gradient Descent used?	Linear regression, logistic regression and neural network training.

Important formulas to remember

1. Original function

$$f(x) = (x+3)^2$$

2. Derivative

$$f'(x) = 2(x+3)$$

3. Gradient Descent update rule

$$x_{new} = x_{old} - \alpha f'(x)$$

4. Final theoretical answer

$$x = -3, \quad y = 0$$

Final conclusion for your practical journal

The Gradient Descent Algorithm was successfully implemented to find the minimum value of the function $y = (x+3)^2$, starting from $x = 2$. The derivative of the function was calculated and used to update the x value in each iteration. A learning rate of 0.1 and a maximum of 25 iterations were used. The algorithm gradually approached the minimum value and obtained x = -2.9811 and y = 0.0003568. Thus, the algorithm successfully demonstrated convergence towards the theoretical minimum at x = -3.

Test your understanding

Try answering these three questions to check whether you understand the practical.

Gradient Descent mini quiz 3 questions

1. What is the derivative of $(x + 3)^2$?

- ○ x + 3
- ○ 2(x + 3)
- ○ 2x + 3
- ○ $(x + 3)^3$

2. What is the purpose of the learning rate?

- ○ To count iterations
- ○ To calculate the derivative
- ○ To control the step size
- ○ To plot the graph

3. What is the theoretical minimum of the given function?

- ○ x = 2
- ○ x = 0
- ○ x = 3
- ○ x = -3

Submit answers

2026-10-31 16:21:39 **You**

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
 accuracy_score,
 confusion_matrix,
 precision_score,
 recall_score,
 f1_score,
 classification_report
)
df = pd.read_csv("diabetes.csv")
df.head()
 Pregnancies Glucose BloodPressure SkinThickness Insulin
BMI \
0 6 148 72 35 0 33.6
1 1 85 66 29 0 26.6
2 8 183 64 0 0 23.3
3 1 89 66 23 94 28.1
4 0 137 40 35 168 43.1
 Pedigree Age Outcome
0 0.627 50 1
1 0.351 31 0
2 0.672 32 1
3 0.167 21 0
4 2.288 33 1
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 # Column Non-Null Count Dtype
--- ------ -------------- -----
 0 Pregnancies 768 non-null int64
 1 Glucose 768 non-null int64
 2 BloodPressure 768 non-null int64
 3 SkinThickness 768 non-null int64
 4 Insulin 768 non-null int64
 5 BMI 768 non-null float64
 6 Pedigree 768 non-null float64
 7 Age 768 non-null int64
 8 Outcome 768 non-null int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
df.describe()
 Pregnancies Glucose BloodPressure SkinThickness
Insulin \
count 768.000000 768.000000 768.000000 768.000000
768.000000
mean 3.845052 120.894531 69.105469 20.536458
79.799479
std 3.369578 31.972618 19.355807 15.952218
115.244002
min 0.000000 0.000000 0.000000 0.000000
0.000000
25% 1.000000 99.000000 62.000000 0.000000
0.000000
50% 3.000000 117.000000 72.000000 23.000000
30.500000
75% 6.000000 140.250000 80.000000 32.000000
127.250000
max 17.000000 199.000000 122.000000 99.000000
846.000000
 BMI Pedigree Age Outcome
count 768.000000 768.000000 768.000000 768.000000
mean 31.992578 0.471876 33.240885 0.348958
std 7.884160 0.331329 11.760232 0.476951
min 0.000000 0.078000 21.000000 0.000000
25% 27.300000 0.243750 24.000000 0.000000
50% 32.000000 0.372500 29.000000 0.000000
75% 36.600000 0.626250 41.000000 1.000000
max 67.100000 2.420000 81.000000 1.000000
cols = ["Glucose", "BloodPressure", "SkinThickness",
 "Insulin", "BMI"]
df[cols] = df[cols].replace(0, np.nan)
for col in cols:
 df[col] = df[col].fillna(df[col].median())
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
X_train, X_test, y_train, y_test = train_test_split(
 X,
 y,
 test_size=0.2,
 random_state=42,
 stratify=y
```

```
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
KNeighborsClassifier(n_neighbors=5)
KNeighborsClassifier()
knn = KNeighborsClassifier()
param_grid = {
 "n_neighbors": range(1, 31),
 "weights": ["uniform", "distance"],
 "metric": ["euclidean", "manhattan"]
}
grid = GridSearchCV(
 knn,
 param_grid,
 cv=5,
 scoring="recall",
 n_jobs=-1
)
grid.fit(X_train_scaled, y_train)
GridSearchCV(cv=5, estimator=KNeighborsClassifier(), n_jobs=-1,
 param_grid={'metric': ['euclidean', 'manhattan'],
 'n_neighbors': range(1, 31),
 'weights': ['uniform', 'distance']},
 scoring='recall')
print(grid.best_params_)
{'metric': 'euclidean', 'n_neighbors': 17, 'weights': 'distance'}
print(grid.best_score_)
0.6167220376522702
best_knn = grid.best_estimator_
y_pred = best_knn.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
Accuracy: 0.7467532467532467
precision = precision_score(y_test, y_pred)
print("Precision:", precision)
Precision: 0.6666666666666666
recall = recall_score(y_test, y_pred)
print("Recall:", recall)
Recall: 0.5555555555555556
f1 = f1_score(y_test, y_pred)
```

```
print("F1 Score:", f1)
F1 Score: 0.6060606060606061
error_rate = 1 - accuracy
print("Error Rate:", error_rate)
Error Rate: 0.2532467532467533
cm = confusion_matrix(y_test, y_pred)
print(cm)
[[85 15]
 [24 30]]
import seaborn as sns
import matplotlib.pyplot as plt
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
print(classification_report(y_test, y_pred))
 precision recall f1-score support
 0 0.78 0.85 0.81 100
 1 0.67 0.56 0.61 54
 accuracy 0.75 154
 macro avg 0.72 0.70 0.71 154
weighted avg 0.74 0.75 0.74 154
```

ChatGPT gpt-6-mini

Practical: Diabetes Prediction using K-Nearest Neighbors (KNN)

Your practical uses the **K-Nearest Neighbors (KNN) classification algorithm** to predict whether a person has diabetes or not, based on medical information such as glucose level, blood pressure, BMI, age, etc.

I'll explain it in simple language, section by section, so you can understand the code and answer viva questions.

1. Aim of the practical

To build a machine learning model using the KNN algorithm to predict diabetes and evaluate its performance using accuracy, precision, recall, F1-score, and a confusion matrix.

2. Understanding the dataset

The dataset is `diabetes.csv`.

It contains 768 records and 9 columns.

Column	Meaning
Pregnancies	Number of times pregnant
Glucose	Blood glucose level
BloodPressure	Blood pressure
SkinThickness	Skin thickness
Insulin	Insulin level
BMI	Body Mass Index
Pedigree	Diabetes pedigree function (family-history-related measure)
Age	Person's age
Outcome	Diabetes result (0 or 1)

Outcome is the target column:

- `0` = No diabetes
- `1` = Diabetes

3. Explanation of the code

Step 1: Importing libraries

```python
import pandas as pd
import numpy as np
```

- `pandas`: Used to load and manipulate datasets.
- `numpy`: Used for numerical operations and handling missing values.

```python
from sklearn.model_selection import train_test_split, GridSearchCV
```

- `train_test_split`: Divides data into training and testing sets.

- `GridSearchCV` : Finds the best parameters for the KNN model.

```python
from sklearn.preprocessing import StandardScaler
```

- StandardScaler scales the features so that they are on a comparable scale.

```python
from sklearn.neighbors import KNeighborsClassifier
```

Imports the KNN classification algorithm.

```python
from sklearn.metrics import (
 accuracy_score, confusion_matrix, precision_score,
 recall_score, f1_score, classification_report
)
```

These functions evaluate the model's performance.

Step 2: Load the dataset

```python
df = pd.read_csv("diabetes.csv")
df.head()
```

- `read_csv()` loads the CSV file.
- `df.head()` displays the first five rows.

Step 3: Dataset information

```python
df.info()
```

Displays:

- Number of rows and columns.
- Column names.
- Data types.
- Non-null values.

Your dataset has 768 rows and 9 columns, with no missing values according to `info()`.

```
</> Python

df.describe()
```

Displays statistical information such as:

- Mean
- Standard deviation
- Minimum
- Maximum
- 25%, 50%, and 75% quartiles.

Important: Even though there are no missing values, some columns contain zero values that may represent missing medical measurements.

Step 4: Handling zero values

```
</> Python

cols = ["Glucose", "BloodPressure", "SkinThickness",
        "Insulin", "BMI"]

df[cols] = df[cols].replace(0, np.nan)
```

This replaces zero values with `NaN` (missing values) in the selected columns.

Why?

Because values such as zero glucose, zero blood pressure, or zero BMI are generally not realistic measurements for this dataset.

```
</> Python

for col in cols:
    df[col] = df[col].fillna(df[col].median())
```

This replaces missing values with the median of that column.

Why median?

Median is less affected by extreme values (outliers) than the mean.

Step 5: Separating input and output

```python
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

- `X` contains input features such as age, glucose, BMI, etc.
- `y` contains the target/output column, Outcome.

`axis=1` means we are dropping a column.

Step 6: Splitting the dataset

```python
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

This divides the dataset into two parts.

- 80% training data = 614 records.
- 20% testing data = 154 records.

Important parameters:

- `test_size=0.2` : Uses 20% for testing.
- `random_state=42` : Ensures the same split each time.
- `stratify=y` : Maintains a similar proportion of diabetic and non-diabetic records in both sets.

Step 7: Feature scaling

```python
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

StandardScaler converts features into a common scale.

Why is this important?

Suppose:

- Age ranges from 21 to 81.
- Insulin ranges from 0 to 846.

Insulin has much larger numerical values. Without scaling, it could dominate the distance calculation in KNN.

`fit_transform()` learns scaling parameters from training data and transforms it.

`transform()` applies the same scaling to test data.

Step 8: Creating the KNN model

```python
knn = KNeighborsClassifier()
```

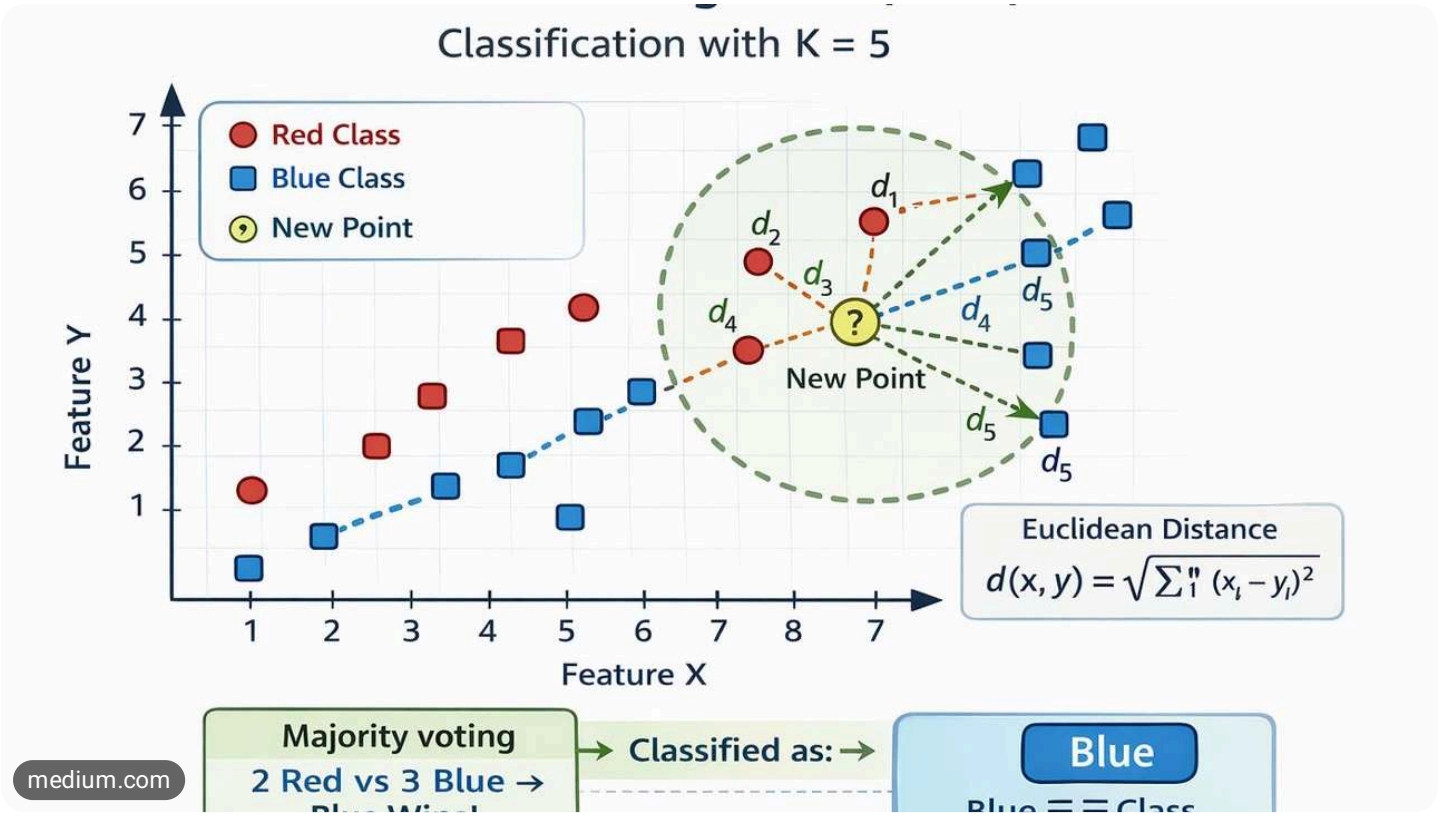
Creates a KNN classification model.

How does KNN work?

KNN predicts the class of a new data point by looking at its nearest neighbours.

For example, if K = 5:

- The algorithm finds the 5 nearest records.
- It checks their classes.
- The majority class becomes the prediction.



Step 9: Hyperparameter tuning

```python
param_grid = {
    "n_neighbors": range(1, 31),
    "weights": ["uniform", "distance"],
    "metric": ["euclidean", "manhattan"]
}
```

This defines the different parameter combinations to test.

Parameter	Meaning
n_neighbors	Number of nearest neighbours (1 to 30)
weights	How neighbours contribute to prediction
uniform	Every neighbour has equal importance
distance	Closer neighbours get more importance
euclidean	Straight-line distance
manhattan	Sum of absolute differences

```python
```

```
grid = GridSearchCV(
    knn,
    param_grid,
    cv=5,
    scoring="recall",
    n_jobs=-1
)
```

- `GridSearchCV` : Tests different combinations.
- `cv=5` : Uses 5-fold cross-validation.
- `scoring="recall"` : Selects the combination with the best recall.
- `n_jobs=-1` : Uses all available CPU cores.

```
</> Python

grid.fit(X_train_scaled, y_train)
```

Trains and tests the different parameter combinations.

Step 10: Finding the best parameters

```
</> Python

print(grid.best_params_)
```

Your output:

```
</> Python

{
 'metric': 'euclidean',
 'n_neighbors': 17,
 'weights': 'distance'
}
```

This means the best model uses:

- 17 nearest neighbours.
- Euclidean distance.
- Distance-based weighting.

```python
print(grid.best_score_)
```

Output:

```
0.6167220376522702
```

This is the best average recall score obtained during 5-fold cross-validation on the training data.

Step 11: Making predictions

```python
best_knn = grid.best_estimator_

y_pred = best_knn.predict(X_test_scaled)
```

- `best_estimator_` selects the best KNN model.
- `predict()` predicts the diabetes outcome for the test data.

Step 12: Evaluating the model

1. Accuracy

```python
accuracy = accuracy_score(y_test, y_pred)
```

Output:

```
Accuracy: 0.74675
```

Accuracy measures how many predictions are correct.

Your model accuracy is approximately **74.68%**.

2. Precision

```
</> Python

precision = precision_score(y_test, y_pred)
```

Output:

```
</> Plain text

Precision: 0.6667
```

Precision tells us how many predicted diabetic cases were actually diabetic.

Your precision is approximately **66.67%**.

3. Recall

```
</> Python

recall = recall_score(y_test, y_pred)
```

Output:

```
</> Plain text

Recall: 0.5556
```

Recall tells us how many actual diabetic cases were correctly identified.

Your recall is approximately **55.56%**.

4. F1 Score

```
</> Python

f1 = f1_score(y_test, y_pred)
```

Output:

```
Plain text

F1 Score: 0.6061
```

F1-score is the harmonic mean of precision and recall.

5. Error Rate

```
Python

error_rate = 1 - accuracy
```

Output:

```
Plain text

Error Rate: 0.2532
```

Error rate tells us the percentage of incorrect predictions.

Your model's error rate is approximately **25.32%**.

4. Confusion matrix

```
Python

cm = confusion_matrix(y_test, y_pred)
print(cm)
```

Output:

```
Plain text

[[85 15]
 [24 30]]
```

Your confusion matrix

85

True Negative (TN)

Correctly predicted no diabetes.

15

False Positive (FP)

Predicted diabetes, but actually no diabetes.

24

False Negative (FN)

Actual diabetes, but predicted no diabetes.

30

True Positive (TP)

Correctly predicted diabetes.

Remember this arrangement:

Actual / Predicted	No diabetes (0)	Diabetes (1)
No diabetes (0)	85	15
Diabetes (1)	24	30

Step 13: Visualizing the confusion matrix

```python
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

- `seaborn` : Used for data visualization.
- `heatmap()` : Displays the confusion matrix as a colored grid.
- `annot=True` : Shows the numbers inside the cells.
- `fmt="d"` : Displays integer values.
- `cmap="Blues"` : Uses blue shades.

Step 14: Classification report

```python
print(classification_report(y_test, y_pred))
```

This displays precision, recall, F1-score, and support for both classes.

- Class 0: No diabetes.
- Class 1: Diabetes.
- Support: Number of actual records in each class.

Your test set contains 100 non-diabetic and 54 diabetic records.

5. Final results

Evaluation metric	Your result
Accuracy	74.68%
Precision	66.67%
Recall	55.56%
F1-score	60.61%
Error rate	25.32%

Important observation: The model correctly identifies only 30 out of 54 actual diabetic cases. It misses 24 diabetic cases. For a medical screening application, this recall is relatively low, so the model should not be used as a real-world diagnostic tool without further validation and improvement.

6. Important viva questions and answers

Q1. What is KNN?

KNN stands for K-Nearest Neighbors. It is a supervised machine learning algorithm used for classification and regression.

Q2. Why is KNN called a lazy learning algorithm?

Because it does not build a traditional model during training. It stores the training data and performs calculations when making predictions.

Q3. What is the value of K in your practical?

The best value of K is 17.

Q4. Why do we use StandardScaler?

To bring all input features to a comparable scale so that large-valued features do not dominate distance calculations.

Q5. What is GridSearchCV?

It is a technique used to find the best hyperparameters by testing different combinations using cross-validation.

Q6. What is cross-validation?

It divides the training data into multiple parts to evaluate a model across different training and validation splits.

Q7. What is the difference between precision and recall?

- Precision: Out of predicted positive cases, how many are actually positive?
- Recall: Out of actual positive cases, how many did the model correctly identify?

Q8. What is a confusion matrix?

It is a table used to evaluate classification models using TP, TN, FP, and FN.

Q9. What is Euclidean distance?

It is the straight-line distance between two points.

Q10. Why did you replace zero values with the median?

Because zero values in some medical columns may represent missing measurements, and the median is less affected by outliers.

Q11. What is the target variable?

`Outcome`.

Q12. What is the difference between classification and regression?

Classification predicts categories, while regression predicts continuous numerical values.

Q13. What is the purpose of `random_state=42`?

It ensures reproducibility by making the random data split consistent.

Q14. What is the purpose of `stratify=y`?

It maintains the class distribution in training and testing data.

Q15. Why is recall important in diabetes prediction?

Because low recall means more actual diabetic cases may be missed.

7. Conclusion for your practical journal

The K-Nearest Neighbors classification algorithm was successfully implemented to predict diabetes using the diabetes dataset. Data preprocessing, missing-value handling, feature scaling, and hyperparameter tuning using GridSearchCV were performed. The model achieved an accuracy of approximately 74.68%, precision of 66.67%, recall of 55.56%, and F1-score of 60.61%. The performance was evaluated using a confusion matrix and classification report.

8. Test your understanding

Mini viva quiz 0/5 answered

1. Which algorithm is used in this practical?

- ○ Linear Regression
- ○ K-Nearest Neighbors
- ○ Decision Tree
- ○ Random Forest

2. What does Outcome = 1 represent?

- ○ No diabetes
- ○ Diabetes
- ○ Missing value
- ○ Age

3. Why do we use StandardScaler?

- ○ Remove rows
- ○ Scale features
- ○ Predict output
- ○ Create graphs

4. What is the best K value found by GridSearchCV?

- ○ 5
- ○ 10
- ○ 17

30

5. What does recall measure?

Correct predictions overall

Correct positive predictions among actual positives

Incorrect predictions

Training time

Check answers

2026-10-31 16:22:49 **You**

Practical - 4: Clustering
Implement K-Means and hierarchical clustering on
sales_data_sample.csv and determine
a suitable number of clusters using the elbow method.
Step 1 : Importing Libraries

```
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
```

Step 2 : Load Dataset

```
df = pd.read_csv("sales_data_sample.csv", encoding="latin1",
on_bad_lines="skip")
print("Dataset shape:", df.shape)
print(df.head())
```

Step 3 : Select numerical features for clustering

```
numeric_features = df.select_dtypes(include="number").copy()
numeric_features = numeric_features.replace([float("inf"), float("-
inf")],
pd.NA).dropna()
print("\nNumerical features used for clustering:")
print(numeric_features.columns.tolist())
print(numeric_features.head())
```

```
Dataset shape: (2823, 25)
 ORDERNUMBER QUANTITYORDERED PRICEEACH
ORDERLINENUMBER SALES \
0 10107 30 95.70 2 2871.00
1 10121 34 81.35 5 2765.90
2 10134 41 94.74 2 3884.34
3 10145 45 83.26 6 3746.70
4 10159 49 100.00 14 5205.27
 ORDERDATE STATUS QTR_ID MONTH_ID YEAR_ID ... \
0 2/24/2003 0:00 Shipped 1 2 2003 ...
1 5/7/2003 0:00 Shipped 2 5 2003 ...
2 7/1/2003 0:00 Shipped 3 7 2003 ...
3 8/25/2003 0:00 Shipped 3 8 2003 ...
4 10/10/2003 0:00 Shipped 4 10 2003 ...
 ADDRESSLINE1 ADDRESSLINE2 CITY STATE \
0 897 Long Airport Avenue NaN NYC NY
1 59 rue de l'Abbaye NaN Reims NaN
2 27 rue du Colonel Pierre Avia NaN Paris NaN
3 78934 Hillside Dr. NaN Pasadena CA
4 7734 Strong St. NaN San Francisco CA
 POSTALCODE COUNTRY TERRITORY CONTACTLASTNAME
CONTACTFIRSTNAME DEALSIZE
0 10022 USA NaN Yu Kwai Small
1 51100 France EMEA Henriot Paul Small
2 75508 France EMEA Da Cunha Daniel Medium
3 90003 USA NaN Young Julie Medium
4 NaN USA NaN Brown Julie Medium
[5 rows x 25 columns]
Numerical features used for clustering:
['ORDERNUMBER', 'QUANTITYORDERED', 'PRICEEACH',
'ORDERLINENUMBER', 'SALES',
'QTR_ID', 'MONTH_ID', 'YEAR_ID', 'MSRP']
 ORDERNUMBER QUANTITYORDERED PRICEEACH
ORDERLINENUMBER SALES QTR_ID
\
0 10107 30 95.70 2 2871.00 1
1 10121 34 81.35 5 2765.90 2
2 10134 41 94.74 2 3884.34 3
3 10145 45 83.26 6 3746.70 3
4 10159 49 100.00 14 5205.27 4
 MONTH_ID YEAR_ID MSRP
```

```
0 2 2003 95
1 5 2003 95
2 7 2003 95
3 8 2003 95
4 10 2003 95
Step 4 : Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(numeric_features)
print("Standardized data shape:", X_scaled.shape)
Standardized data shape: (2823, 9)
Step 5 : K-Means clustering
wcss = []
for k in range(1, 11):
 model = KMeans(n_clusters=k, random_state=42, n_init=10)
 model.fit(X_scaled)
 wcss.append(model.inertia_)
plt.figure(figsize=(6, 4))
plt.plot(range(1, 11), wcss, marker="o")
plt.title("Elbow Method")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia (WCSS)")
plt.tight_layout()
plt.savefig("code_4_elbow_method.png")
plt.close()
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters_kmeans = kmeans.fit_predict(X_scaled)
clustered_df = df.loc[numeric_features.index].copy()
clustered_df["KMeans_Cluster"] = clusters_kmeans
plt.figure(figsize=(7, 5))
for cluster_id in sorted(clustered_df["KMeans_Cluster"].unique()):
 cluster_rows = clustered_df[clustered_df["KMeans_Cluster"] ==
cluster_id]
 plt.scatter(cluster_rows["SALES"], cluster_rows["PRICEEACH"],
label=f"Cluster {cluster_id}", alpha=0.7)
plt.title("K-Means Clustering")
plt.xlabel("SALES")
plt.ylabel("PRICEEACH")
plt.legend()
plt.tight_layout()
plt.savefig("code_4_kmeans_clusters.png")
plt.close()
```

```
print("\nElbow WCSS values:", [round(value, 2) for value in wcss])
print("\nK-Means cluster centers:")
print(pd.DataFrame(kmeans.cluster_centers_,
columns=numeric_features.columns))
print("\nK-Means cluster counts:")
print(clustered_df["KMeans_Cluster"].value_counts().sort_index())
print("\nPlots saved: code_4_elbow_method.png,
code_4_kmeans_clusters.png")
Elbow WCSS values: [25407.0, 20090.89, 16909.33, 14818.01,
13541.41,
12547.11, 11743.69, 11067.29, 10529.86, 10085.26]
K-Means cluster centers:
 ORDERNUMBER QUANTITYORDERED PRICEEACH
ORDERLINENUMBER SALES \
0 0.805753 0.154271 0.158539 -0.099048 0.137189
1 -0.263803 0.039507 0.705798 0.015109 0.535064
2 -0.308327 -0.176570 -1.050030 0.060515 -0.809798
 QTR_ID MONTH_ID YEAR_ID MSRP
0 -1.153125 -1.155904 1.181622 0.070180
1 0.509145 0.506268 -0.444983 0.624008
2 0.269399 0.275407 -0.376264 -0.871648
K-Means cluster counts:
KMeans_Cluster
0 734
1 1183
2 906
Name: count, dtype: int64
Plots saved: code_4_elbow_method.png,
code_4_kmeans_clusters.png
Step 6 : Hierarchical clustering
Z = linkage(X_scaled, method="ward")
plt.figure(figsize=(10, 5))
dendrogram(Z, no_labels=True)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.tight_layout()
plt.savefig("code_4_dendrogram.png")
plt.close()
clusters_hier = fcluster(Z, t=3, criterion="maxclust")
clustered_df["Hierarchical_Cluster"] = clusters_hier
```

```
print("\nHierarchical cluster counts:")
print(clustered_df["Hierarchical_Cluster"].value_counts().sort_index()
)
print("Plot saved: code_4_dendrogram.png")
Hierarchical cluster counts:
Hierarchical_Cluster
1 1655
2 478
3 690
Name: count, dtype: int64
Plot saved: code_4_dendrogram.png
```

ChatGPT gpt-6-mini

Practical 4: Clustering using K-Means and Hierarchical Clustering

This practical uses two unsupervised machine learning algorithms, **K-Means Clustering** and **Hierarchical Clustering**, to group sales records into similar categories.

I'll explain the complete code, output, graphs, and viva questions in simple language.

1. Aim of the practical

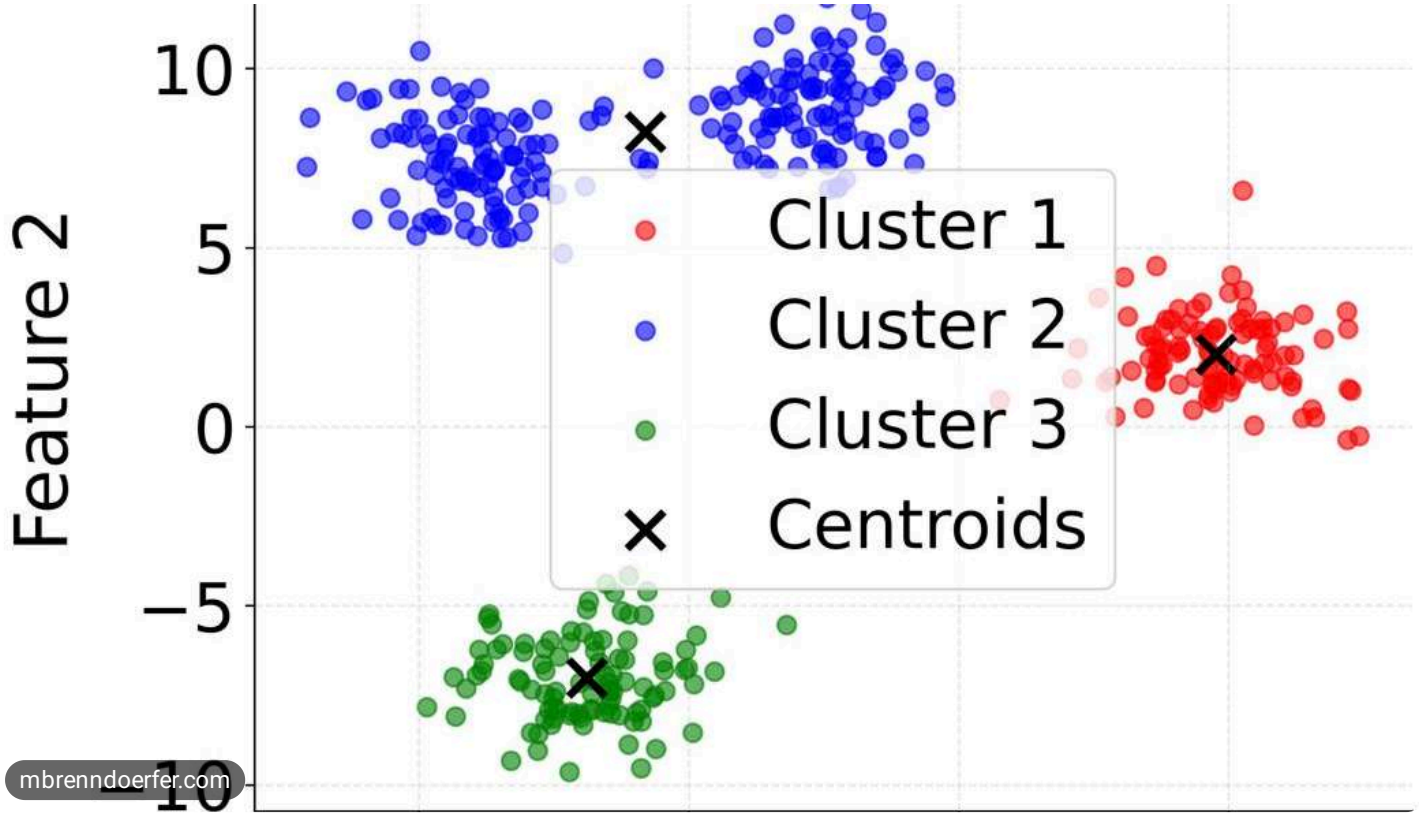
To implement K-Means and Hierarchical Clustering on the `sales_data_sample.csv` dataset and determine a suitable number of clusters using the Elbow Method.

2. What is clustering?

Clustering is an unsupervised machine learning technique used to divide data into groups based on similarities.

For example, a company can group its sales records into:

- Low sales.
- Medium sales.
- High sales.



Important: Clustering does not require predefined output labels. The algorithm finds groups automatically.

3. Explanation of the code

Step 1: Importing libraries

```python
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
```

Explanation:

Library	Purpose
pandas	Loads and manages datasets
matplotlib	Creates graphs
StandardScaler	Standardizes numerical data
KMeans	Performs K-Means clustering

linkage	Performs hierarchical clustering calculations
dendrogram	Creates a hierarchical clustering tree
fcluster	Creates cluster labels from hierarchical results

What is `matplotlib.use("Agg")`?

It selects a non-interactive plotting backend. This allows graphs to be saved as image files without displaying them on screen.

Step 2: Loading the dataset

```
</> Python

df = pd.read_csv(
    "sales_data_sample.csv",
    encoding="latin1",
    on_bad_lines="skip"
)

print("Dataset shape:", df.shape)
print(df.head())
```

Explanation:

- `read_csv()` loads the CSV file.
- `encoding="latin1"` helps read special characters in the dataset.
- `on_bad_lines="skip"` skips incorrectly formatted rows.
- `df.shape` displays the number of rows and columns.
- `df.head()` displays the first five records.

Your output:

```
</> Plain text

Dataset shape: (2823, 25)
```

This means:

- 2823 rows (records).
- 25 columns (features).

Step 3: Selecting numerical features

```python
numeric_features = df.select_dtypes(
    include="number"
).copy()
```

This selects only numerical columns from the dataset.

Why?

Clustering algorithms such as K-Means calculate distances, so they require numerical input.

```python
numeric_features = numeric_features.replace(
    [float("inf"), float("-inf")],
    pd.NA
).dropna()
```

This performs two operations:

1. Replaces positive and negative infinity with missing values.
2. Removes rows containing missing values in the selected numerical columns.

```python
print(numeric_features.columns.tolist())
print(numeric_features.head())
```

Displays the numerical columns and first five records.

Your dataset contains 9 numerical features:

1. ORDERNUMBER
2. QUANTITYORDERED
3. PRICEEACH
4. ORDERLINENUMBER
5. SALES
6. QTR_ID
7. MONTH_ID
8. YEAR_ID
9. MSRP

Viva point: Why do we select numerical columns?

Because clustering algorithms calculate distances between data points.

Step 4: Standardizing the data

```python
scaler = StandardScaler()

X_scaled = scaler.fit_transform(numeric_features)

print("Standardized data shape:", X_scaled.shape)
```

StandardScaler brings all numerical features to a comparable scale.

For example:

- ORDERNUMBER may have values around 10,000.
- SALES may have values around 5,000.
- QUANTITYORDERED may have values around 30.

Without scaling, large numerical values could dominate distance calculations.

The formula is:

$$z = \frac{x - \mu}{\sigma}$$

Where:

- x = original value.
- μ = mean.
- σ = standard deviation.
- z = standardized value.

Output:

```
Standardized data shape: (2823, 9)
```

This means 2823 records and 9 standardized features.

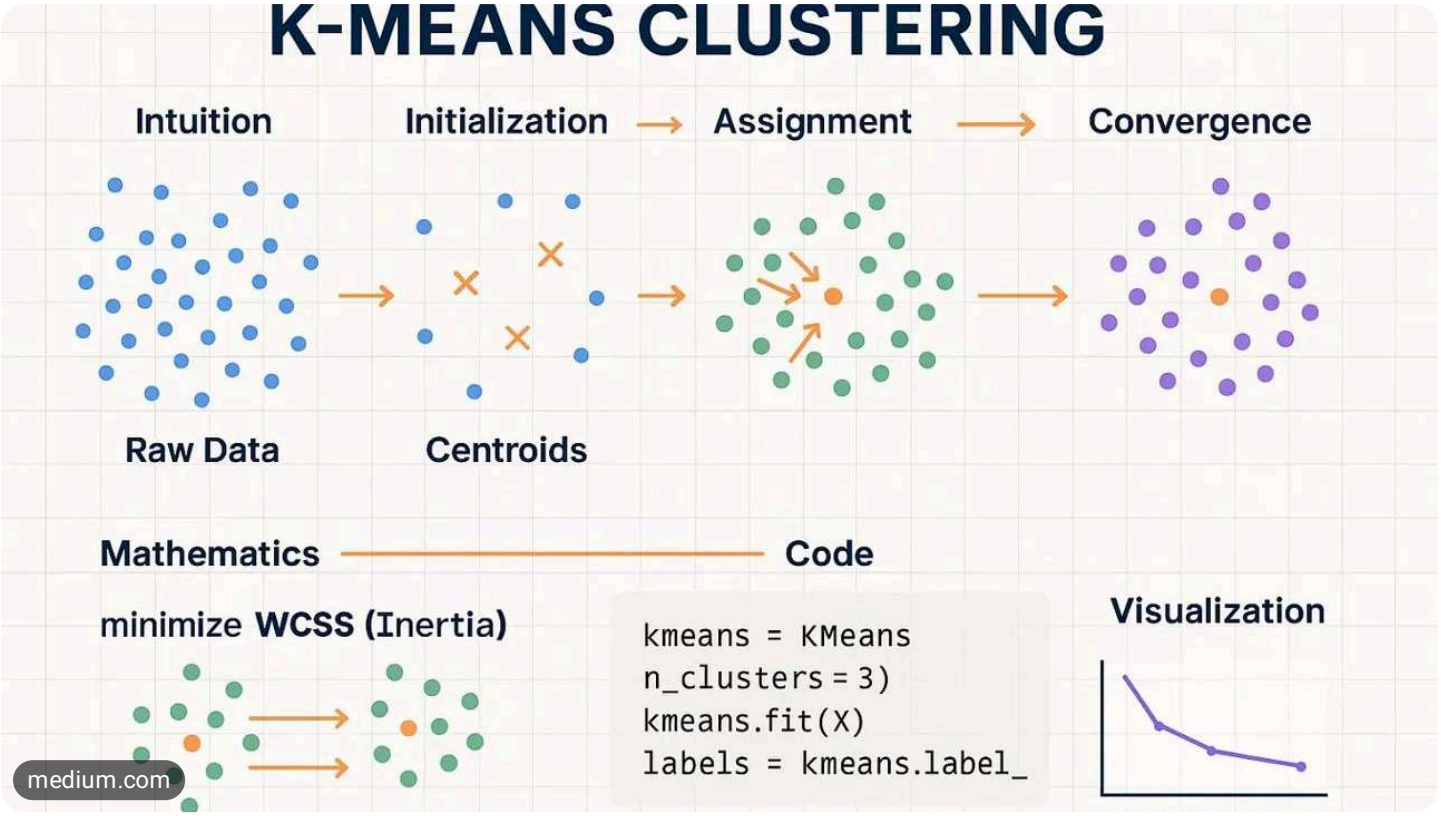
4. Step 5: K-Means clustering

What is K-Means?

K-Means is an unsupervised machine learning algorithm that divides data into K clusters.

It works by:

1. Selecting K initial cluster centers.
2. Assigning each data point to its nearest center.
3. Recalculating the cluster centers.
4. Repeating until the clusters stop changing significantly.



A. Finding the suitable number of clusters using Elbow Method

```python
wcss = []

for k in range(1, 11):
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    model.fit(X_scaled)
```

```
    wcss.append(model.inertia_)
```

Explanation:

- `wcss = []` : Creates an empty list to store WCSS values.
- `range(1, 11)` : Tests cluster numbers from 1 to 10.
- `n_clusters=k` : Specifies the number of clusters.
- `random_state=42` : Makes results reproducible.
- `n_init=10` : Runs K-Means 10 times with different initial centroid selections and keeps the best result.
- `model.fit(X_scaled)` : Trains the clustering algorithm.
- `model.inertia_` : Returns the WCSS value.

What is WCSS?

WCSS stands for Within-Cluster Sum of Squares.

It measures how close data points are to their cluster centers.

Lower WCSS generally means tighter clusters.

B. Drawing the Elbow graph

```python
plt.figure(figsize=(6, 4))

plt.plot(range(1, 11), wcss, marker="o")

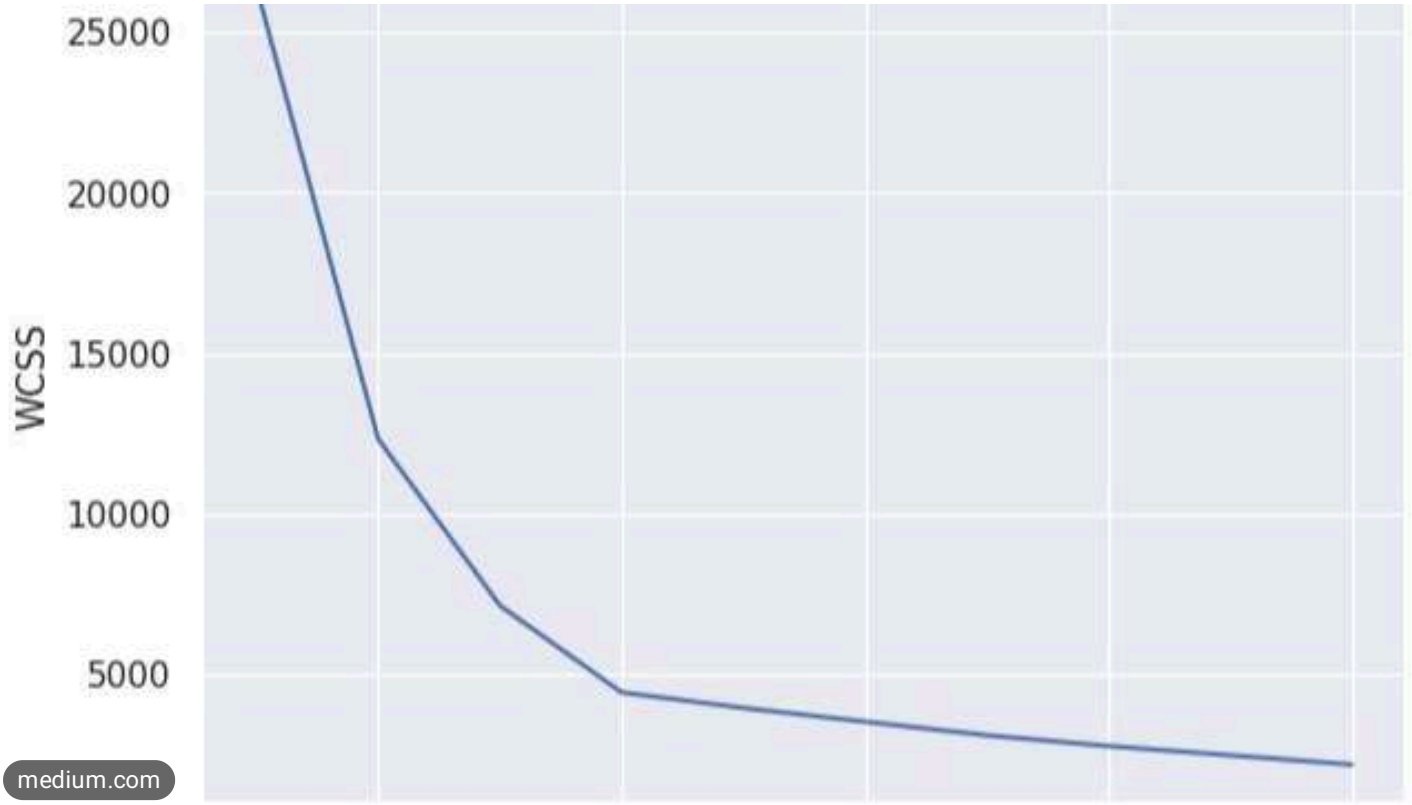
plt.title("Elbow Method")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia (WCSS)")

plt.tight_layout()
plt.savefig("code_4_elbow_method.png")
plt.close()
```

Explanation:

- `figure()` : Creates a graph.
- `plot()` : Draws the WCSS line.
- `marker="o"` : Displays circular points.
- `xlabel()` : Labels the X-axis.
- `ylabel()` : Labels the Y-axis.
- `savefig()` : Saves the graph as an image.

- `close()` : Closes the figure.



How do we select K?

We look for the elbow point: the point where the decrease in WCSS starts becoming slower.

Your program tests K from 1 to 10, and then chooses K = 3 for the final model.

Your WCSS values are:

K	WCSS
1	25407.00
2	20090.89
3	16909.33
4	14818.01
5	13541.41
6	12547.11
7	11743.69
8	11067.29
9	10529.86
10	10085.26

Remember: Your code chooses 3 clusters, but it does not automatically detect the elbow. You should verify that K = 3 is supported by the actual plotted curve.

C. Applying K-Means with 3 clusters

```python
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

clusters_kmeans = kmeans.fit_predict(X_scaled)
```

Explanation:

- `n_clusters=3` : Creates 3 clusters.
- `fit_predict()` : Trains the model and assigns each record a cluster number.
- `clusters_kmeans` : Stores the assigned cluster labels.

D. Adding cluster labels to the dataset

```python
clustered_df = df.loc[numeric_features.index].copy()

clustered_df["KMeans_Cluster"] = clusters_kmeans
```

This creates a copy of the original dataset containing the valid numerical records and adds a new column called `KMeans_Cluster`.

The cluster labels are:

- 0 = Cluster 1
- 1 = Cluster 2
- 2 = Cluster 3

These are simply machine-generated identifiers, not rankings.

E. Visualizing K-Means clusters

```python
```

```
plt.figure(figsize=(7, 5))

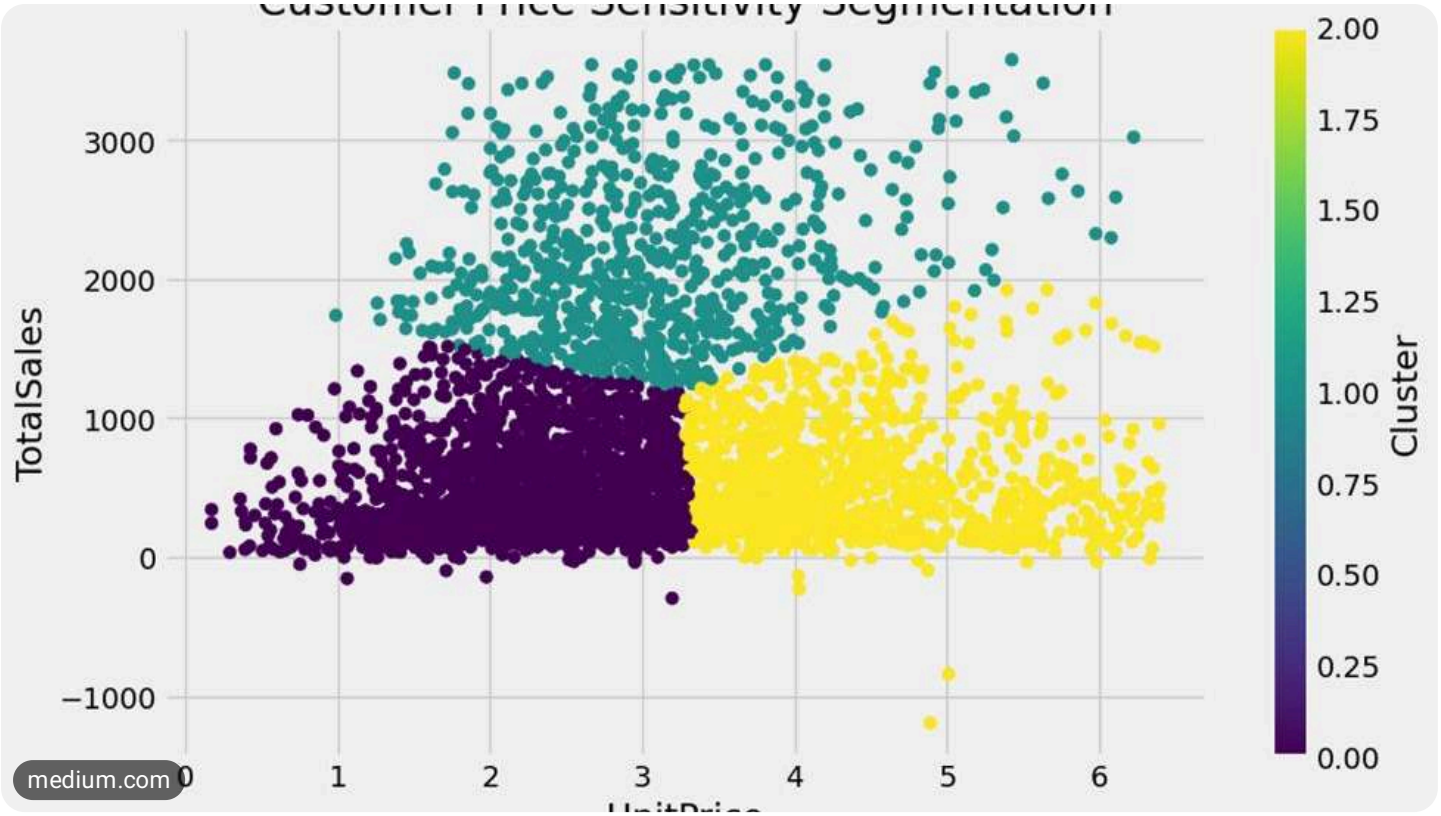
for cluster_id in sorted(
    clustered_df["KMeans_Cluster"].unique()
):
    cluster_rows = clustered_df[
        clustered_df["KMeans_Cluster"] == cluster_id
    ]

    plt.scatter(
        cluster_rows["SALES"],
        cluster_rows["PRICEEACH"],
        label=f"Cluster {cluster_id}",
        alpha=0.7
    )

plt.title("K-Means Clustering")
plt.xlabel("SALES")
plt.ylabel("PRICEEACH")
plt.legend()
plt.tight_layout()
plt.savefig("code_4_kmeans_clusters.png")
plt.close()
```

Explanation:

- `unique()` : Finds all unique cluster numbers.
- `cluster_rows` : Selects records belonging to one cluster.
- `scatter()` : Creates a scatter plot.
- X-axis = Sales.
- Y-axis = Price each.
- `label` : Gives each cluster a name.
- `alpha=0.7` : Controls transparency.
- `legend()` : Displays the cluster labels.



F. Printing WCSS, centers and cluster counts

```python
print("Elbow WCSS values:", [round(value, 2) for value in wcss])
```

Prints WCSS values rounded to two decimal places.

```python
print(pd.DataFrame(
    kmeans.cluster_centers_,
    columns=numeric_features.columns
))
```

Displays the centers of the three clusters.

Because the data was standardized, these centers are expressed in standardized units, not the original sales or price units.

```python
print(
    clustered_df["KMeans_Cluster"].value_counts().sort_index()
```

```
)
```

Your output:

Cluster	Number of records
0	734
1	1183
2	906
Total	**2823**

This tells us how many records belong to each cluster.

5. Step 6: Hierarchical clustering

What is Hierarchical Clustering?

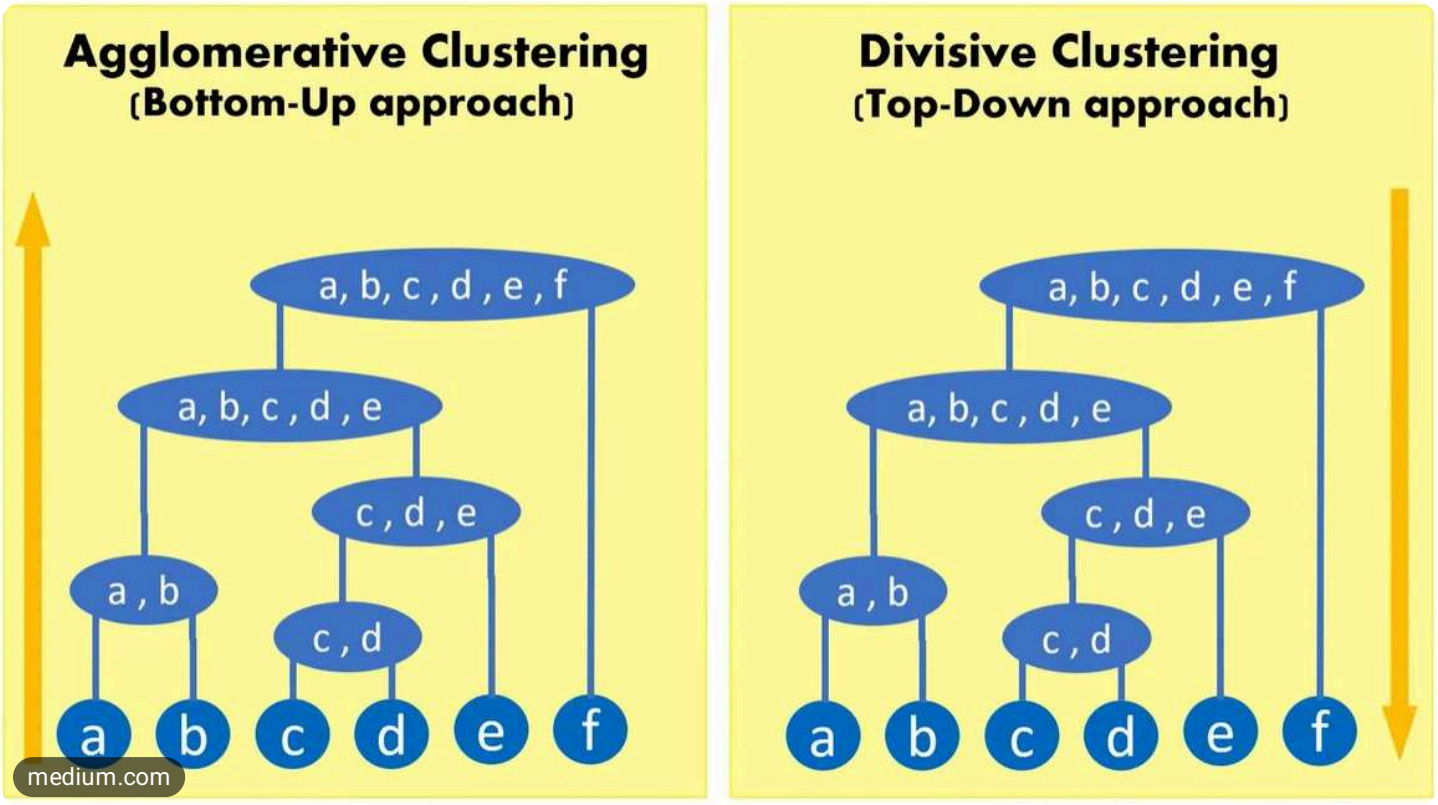
Hierarchical clustering is an unsupervised machine learning technique that creates a hierarchy of clusters.

It groups similar data points step by step.

There are two types:

1. Agglomerative: Starts with individual points and combines them into larger clusters.
2. Divisive: Starts with one large cluster and divides it into smaller clusters.

Your practical uses **Agglomerative Hierarchical Clustering**.



A. Creating the linkage matrix

```python
Z = linkage(X_scaled, method="ward")
```

Explanation:

- `linkage()` performs hierarchical clustering.
- `X_scaled` is the standardized dataset.
- `method="ward"` uses Ward's method.

What is Ward's method?

Ward's method combines clusters in a way that minimizes the increase in within-cluster variance.

In simple words, it tries to combine groups that are similar to each other.

B. Drawing the dendrogram

```python
plt.figure(figsize=(10, 5))

dendrogram(Z, no_labels=True)

plt.title("Hierarchical Clustering Dendrogram")
```

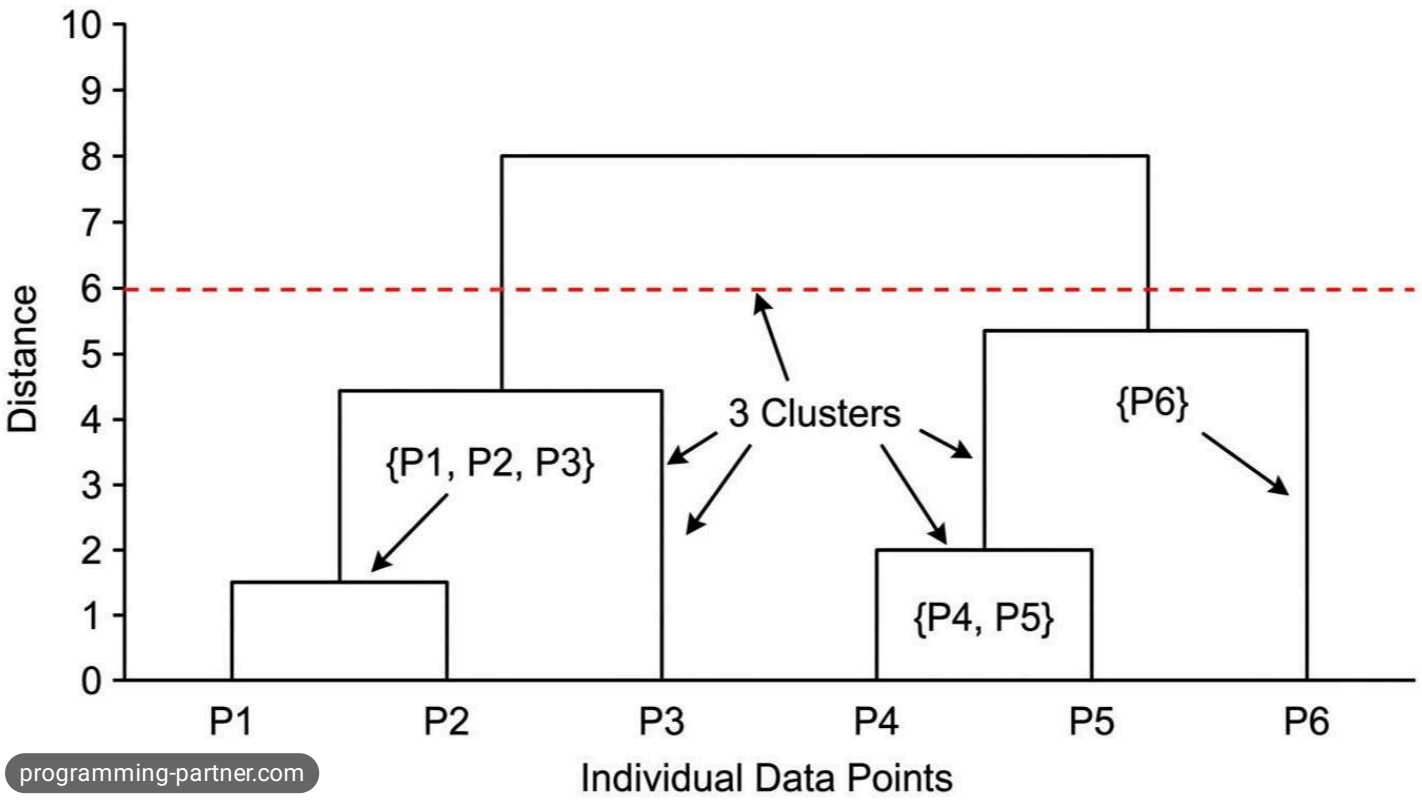
```
plt.xlabel("Samples")
plt.ylabel("Distance")

plt.tight_layout()
plt.savefig("code_4_dendrogram.png")
plt.close()
```

What is a dendrogram?

A dendrogram is a tree-like diagram that shows how data points or clusters are joined together.

- Bottom of the tree: Individual data points.
- Branches: Merging of clusters.
- Height: Distance at which clusters are joined.



`no_labels=True` hides individual sample labels because there are many records.

C. Creating 3 hierarchical clusters

```
</> Python

clusters_hier = fcluster(
    Z,
    t=3,
    criterion="maxclust"
)
```

Explanation:

- `fcluster()` : Converts the hierarchical tree into cluster labels.
- `Z` : Linkage matrix.
- `t=3` : Requests a maximum of 3 clusters.
- `criterion="maxclust"` : Uses the maximum number of clusters as the criterion.

```python
clustered_df["Hierarchical_Cluster"] = clusters_hier
```

Adds hierarchical cluster labels to the dataset.

```python
print(
    clustered_df["Hierarchical_Cluster"].value_counts().sort_index()
)
```

Your output:

Hierarchical cluster	Number of records
1	1655
2	478
3	690
Total	**2823**

The records are divided into three hierarchical clusters.

---

6. Final comparison

Feature	K-Means	Hierarchical
Type	Unsupervised	Unsupervised
Approach	Centroid-based	Tree-based
Number of clusters	Specified before fitting	Can be selected by cutting the tree
Main visualization	Elbow graph	Dendrogram
Output	Cluster labels and centers	Cluster labels and hierarchy

Practical method	`KMeans()`	`linkage()` and `fcluster()`

Important observation: Both algorithms divided your 2823 records into 3 clusters, but the number of records in each cluster is different.

7. Important viva questions and answers

Q1. What is clustering?

Clustering is an unsupervised machine learning technique used to group similar data points together.

Q2. What is K-Means?

K-Means is a clustering algorithm that divides data into K groups based on the distance from cluster centers.

Q3. What is the meaning of K in K-Means?

K represents the number of clusters.

Q4. What is the Elbow Method?

It is a technique used to determine a suitable number of clusters by plotting WCSS against the number of clusters.

Q5. What is WCSS?

WCSS stands for Within-Cluster Sum of Squares. It measures the sum of squared distances between data points and their cluster centers.

Q6. Why do we use StandardScaler?

To standardize features so that large-valued features do not dominate distance calculations.

Q7. What is hierarchical clustering?

It is a clustering technique that creates a tree-like hierarchy of clusters.

Q8. What is a dendrogram?

A dendrogram is a tree diagram that shows how clusters are merged or divided.

Q9. What is Ward's method?

Ward's method merges clusters while minimizing the increase in within-cluster variance.

Q10. What are the two types of hierarchical clustering?

Agglomerative and divisive.

Q11. What is the difference between supervised and unsupervised learning?

- Supervised learning uses labelled data.
- Unsupervised learning uses unlabelled data.

Q12. Why is clustering useful in sales?

It helps companies group customers, identify sales patterns, understand purchasing behaviour, and develop marketing strategies.

Q13. What is a cluster centroid?

It is the center point of a cluster, calculated as the mean of its data points.

Q14. What does `random_state=42` do?

It makes the random initialization reproducible.

Q15. What is `fit_predict()`?

It trains the clustering model and returns a cluster label for every data point.

8. Conclusion for practical journal

The K-Means and Hierarchical Clustering algorithms were successfully implemented on the sales dataset. Numerical features were selected and standardized before clustering. The Elbow Method was used to examine a suitable number of clusters, and K-Means was implemented with 3 clusters. Hierarchical clustering was also performed using Ward's method and visualized using a dendrogram. Both algorithms grouped the 2823 sales records into three clusters, helping identify patterns and similarities in sales data.

---

9. Mini viva test

Check your knowledge 0/5 answered

1. Which type of machine learning is clustering?

- ◯ Supervised
- ◯ Unsupervised
- ◯ Reinforcement
- ◯ Regression

2. What does K represent in K-Means?

- ◯ Number of rows
- ◯ Number of features
- ◯ Number of clusters
- ◯ Number of columns

3. Which method is used to find a suitable K?

- ◯ Elbow Method
- ◯ Linear Method
- ◯ Regression Method
- ◯ Sorting Method

4. What does a dendrogram represent?

- ◯ Bar graph
- ◯ Tree-like cluster structure
- ◯ Pie chart
- ◯ Confusion matrix

5. Which method is used in your hierarchical clustering?

- ◯ Single
- ◯ Complete
- ◯ Ward
- ◯ Average

Submit answers

One important practical point: Your code selects all numerical columns, including `ORDERNUMBER`, `YEAR_ID`, and `MONTH_ID`. This is acceptable for demonstrating clustering, but these fields can influence the grouping based on order sequence and time rather than only sales behaviour. Also, the Elbow Method output alone does not prove that 3 is the optimal K; inspect the saved elbow graph before stating that conclusion.